

Convolutional Neural Networks

Vladislav Goncharenko



MSAI, Fall 2024

Revise

Recurrent
Neural Networks

1. Vanilla RNN
2. Gradient related problems
 - a. Exploding
 - b. Vanishing
3. LSTM
 - a. GRU
4. Multilayer RNNs
 - a. bidirectional

Outline

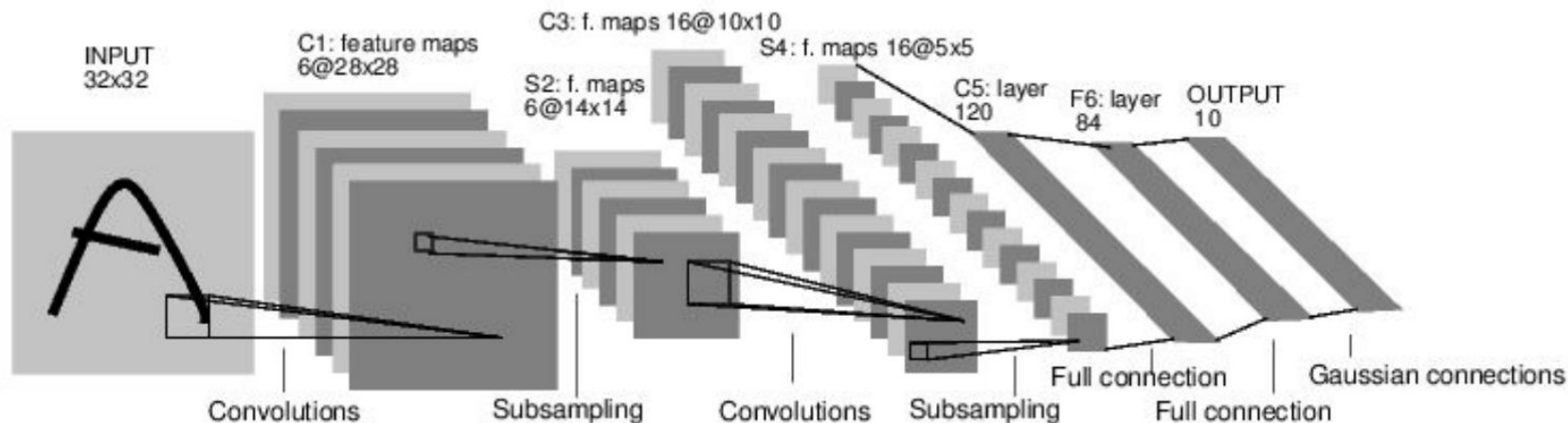
1. Convolutional layer
2. Pooling layers
3. Classification architectures overview

Convolutional layer

girafe
ai

01

Convolutions in Neural Networks

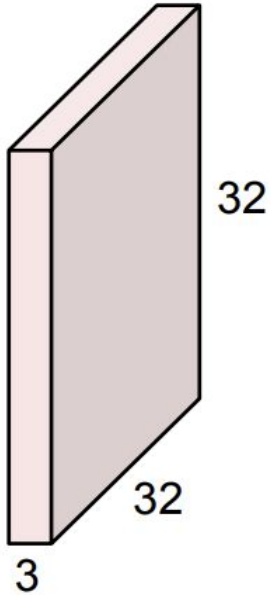


[LeNet-5, LeCun 1998]

Convolutional layer



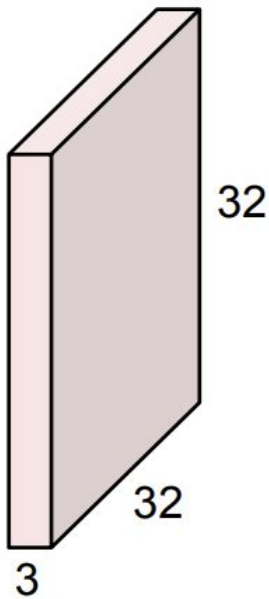
32x32x3 image





Convolutional layer

32x32x3 image



5x5x3 filter

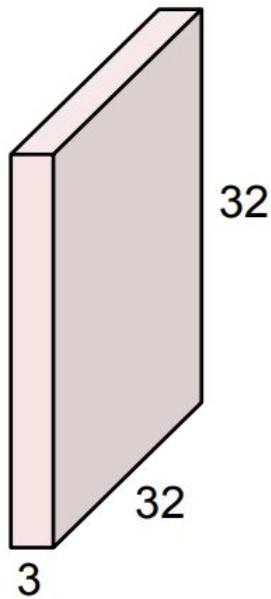


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”



Convolutional layer

32x32x3 image



Filters extend the depth of the original image

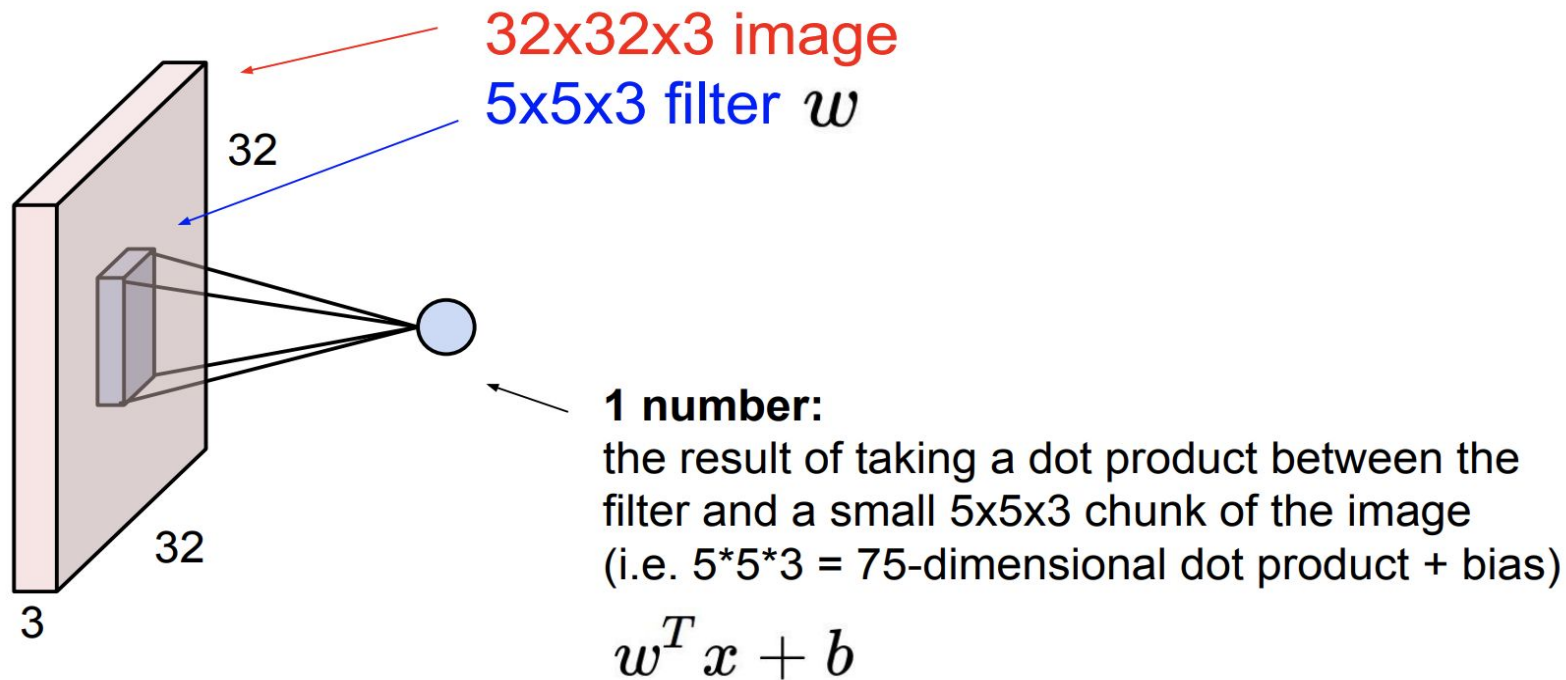
5x5x3 filter



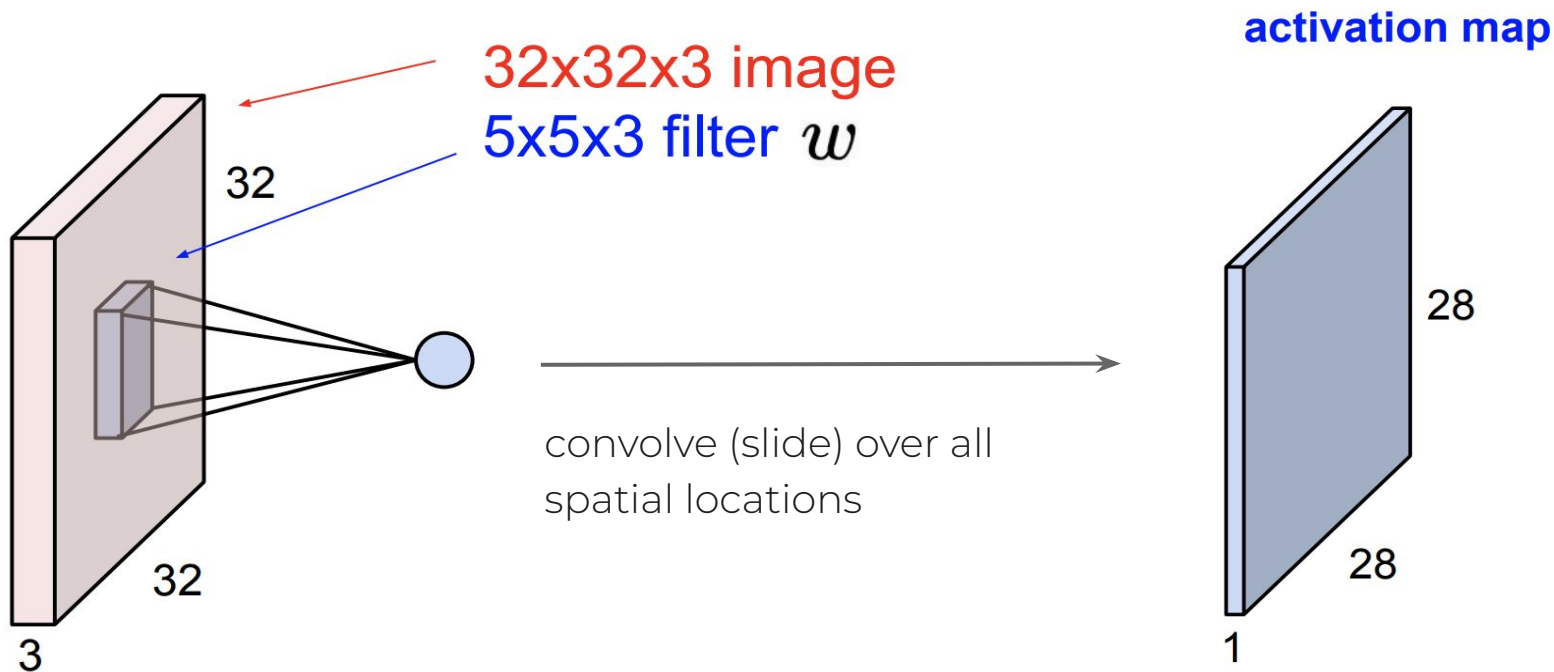
Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”



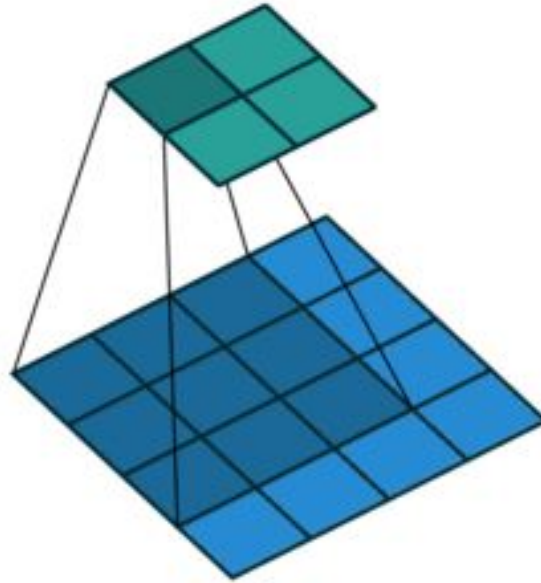
Convolutional layer



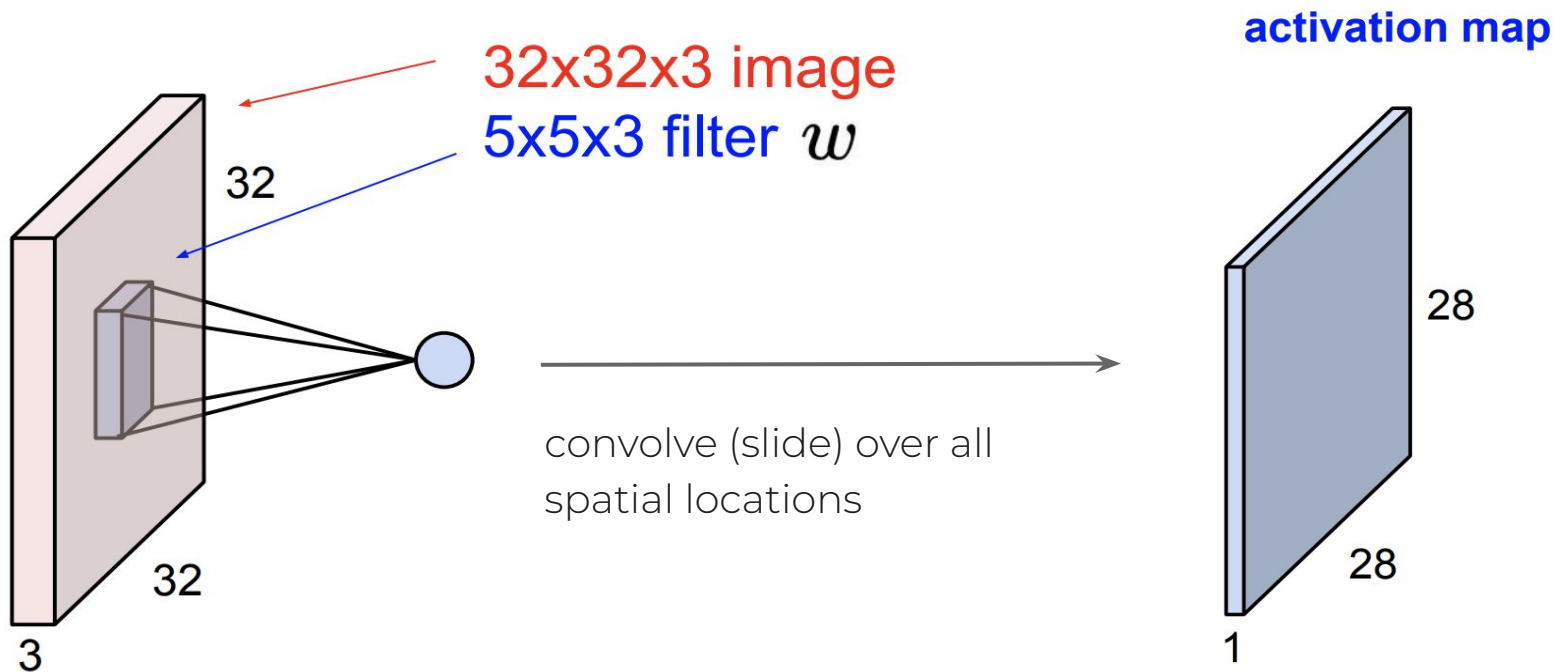
Convolutional layer



Convolutional layer



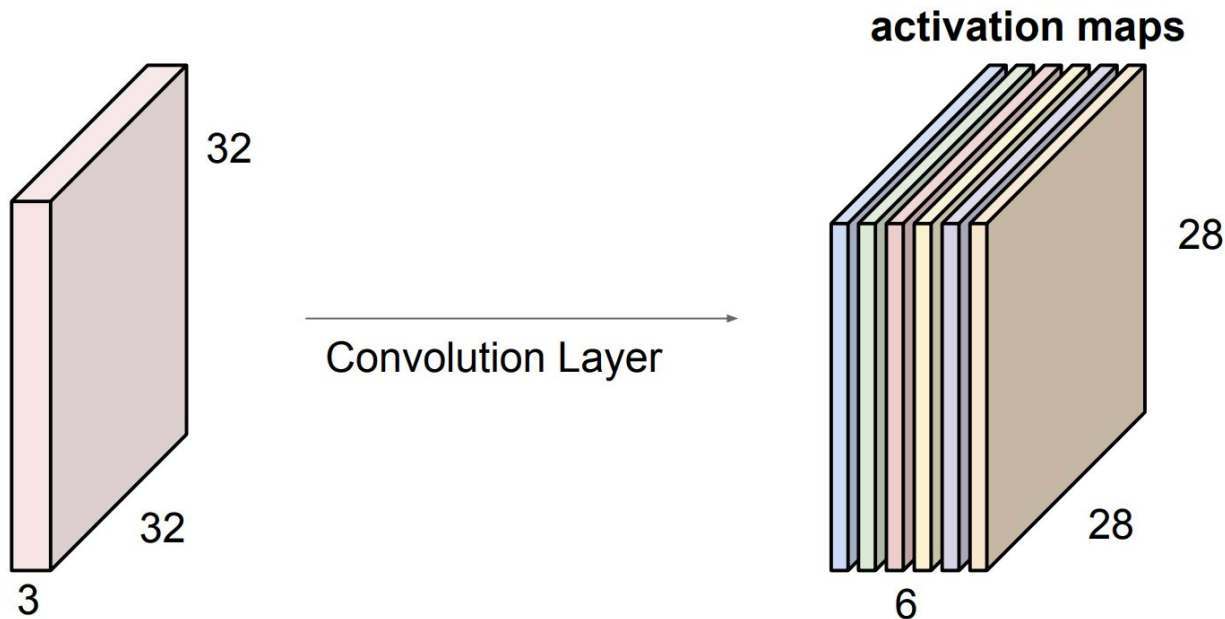
Convolutional layer





Convolutional layer

For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

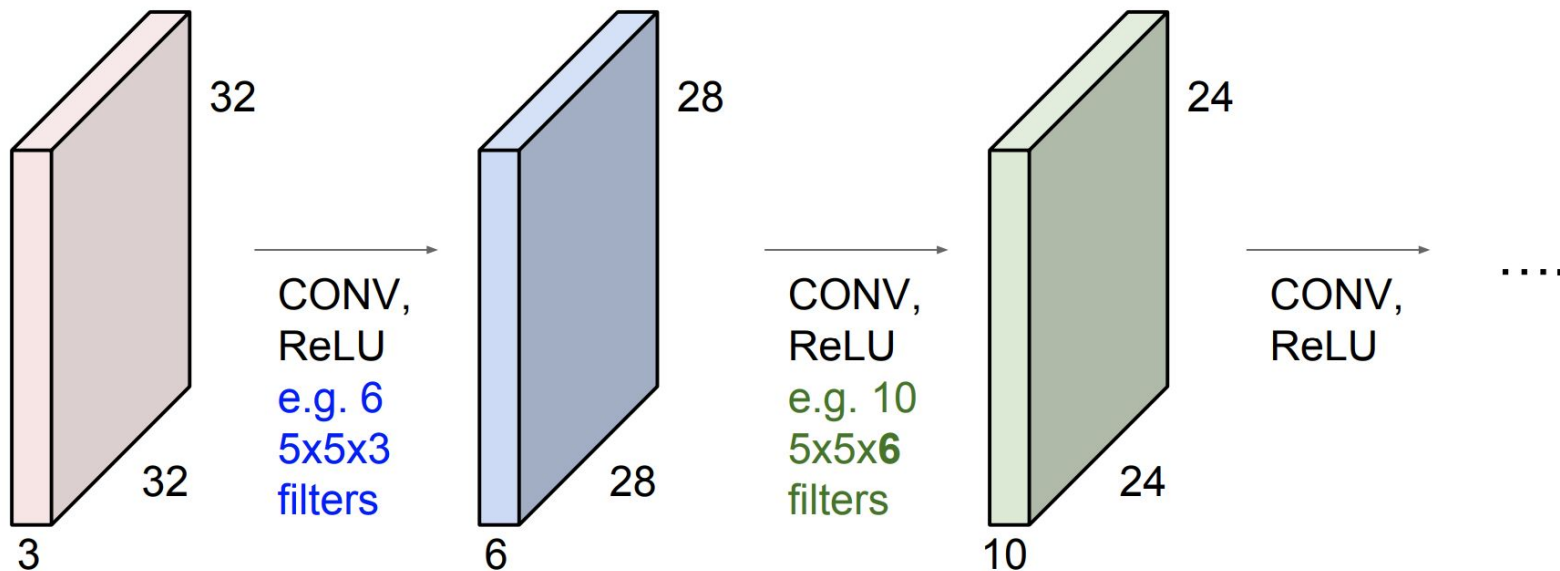


We stack these up to get a “new image” of size 28x28x6!

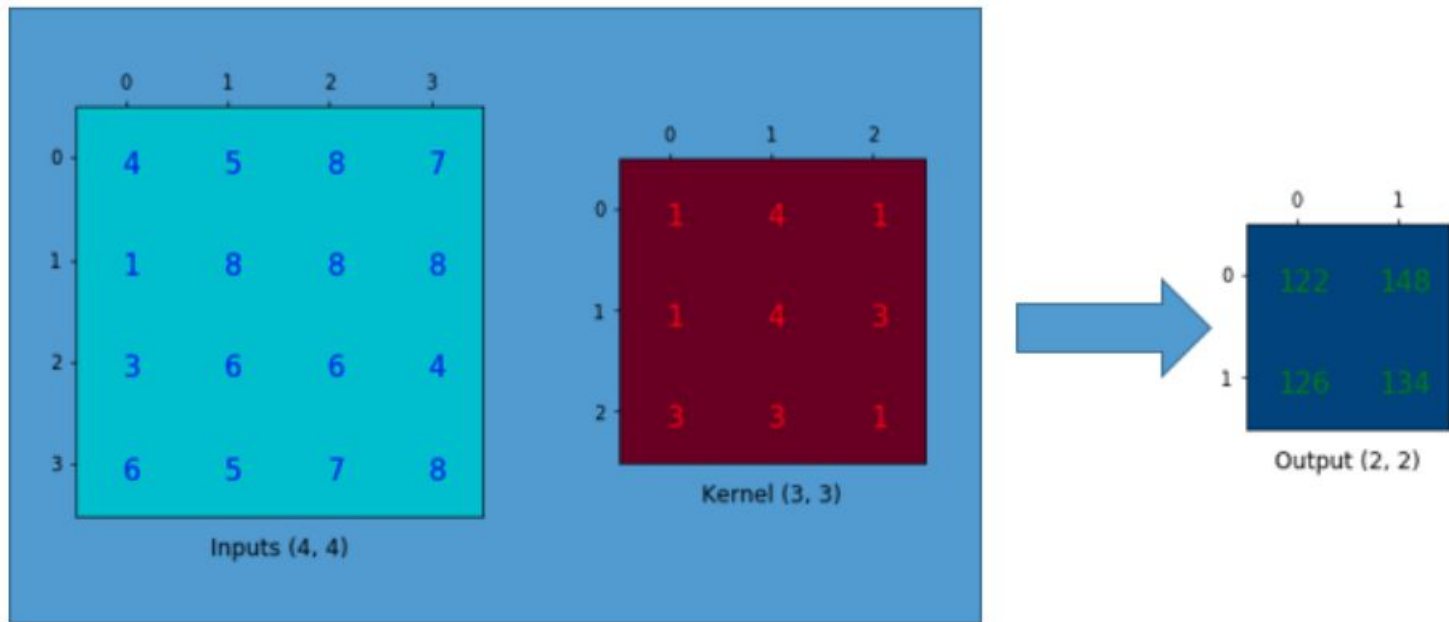


Convolutional layer

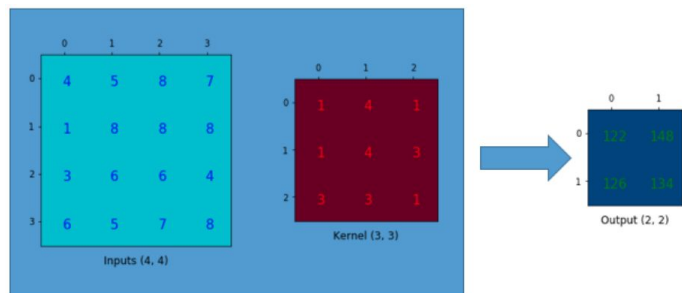
Preview: ConvNet is a sequence of Convolutional Layers, interspersed with activation functions



Regular convolution



Regular convolution: matrix form



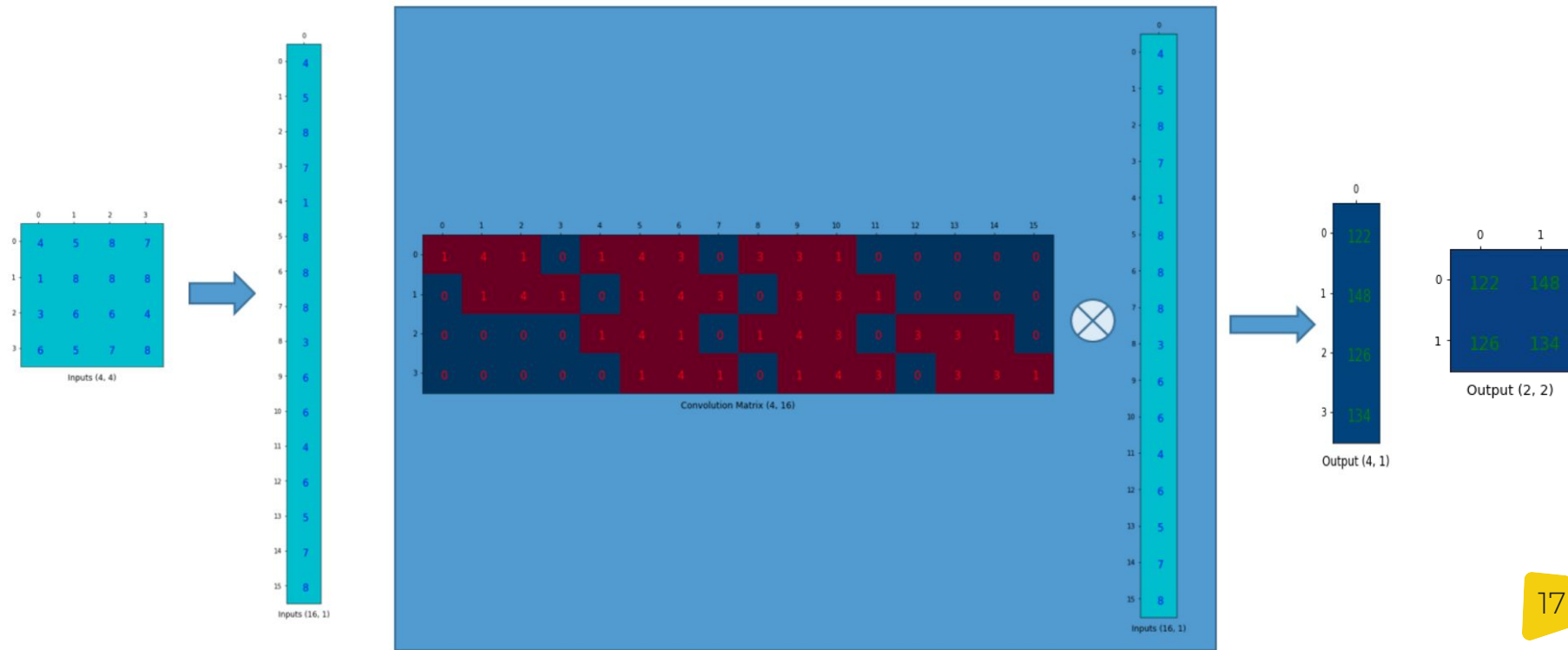
	0	1	2
0	1	4	1
1	1	4	3
2	3	3	1

Kernel (3, 3)

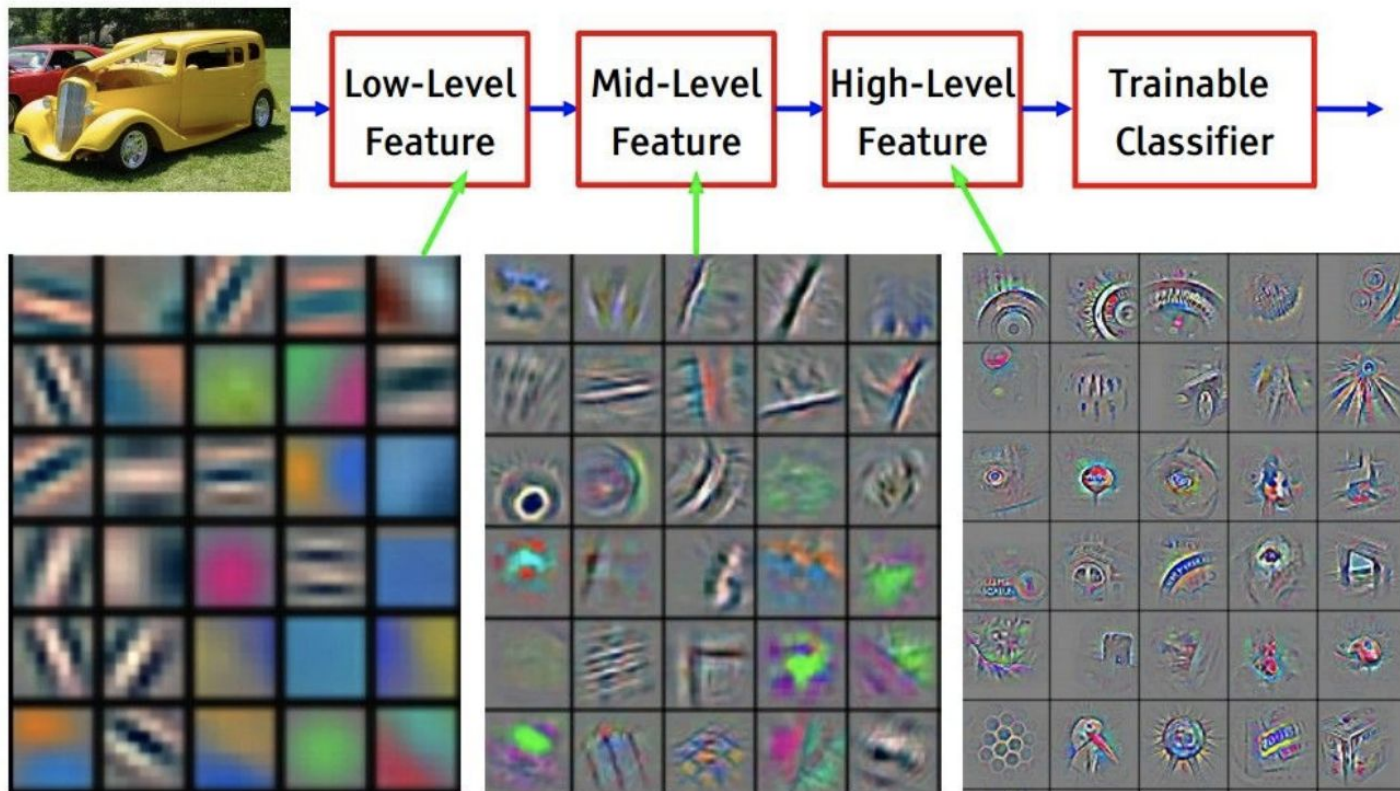
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	4	1	0	1	4	3	0	3	3	1	0	0	0	0	0
1	0	1	4	1	0	1	4	3	0	3	3	1	0	0	0	0
2	0	0	0	0	1	4	1	0	1	4	3	0	3	3	1	0
3	0	0	0	0	0	1	4	1	0	1	4	3	0	3	3	1

Convolution Matrix (4, 16)

Regular convolution: matrix form



Convolutional layer



Deep dream

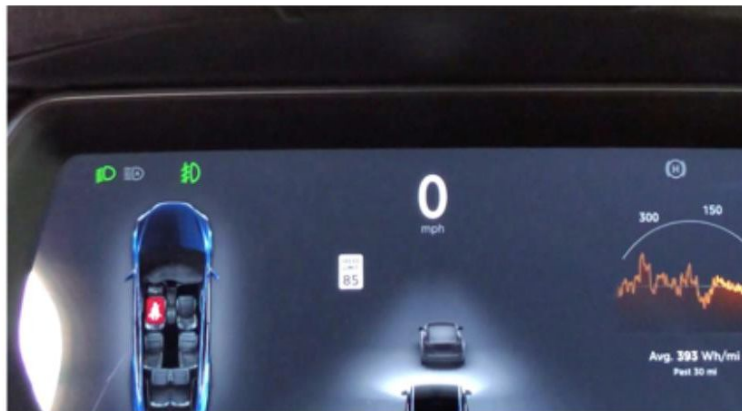
<https://www.youtube.com/watch?v=DgPaCWJL7XI>



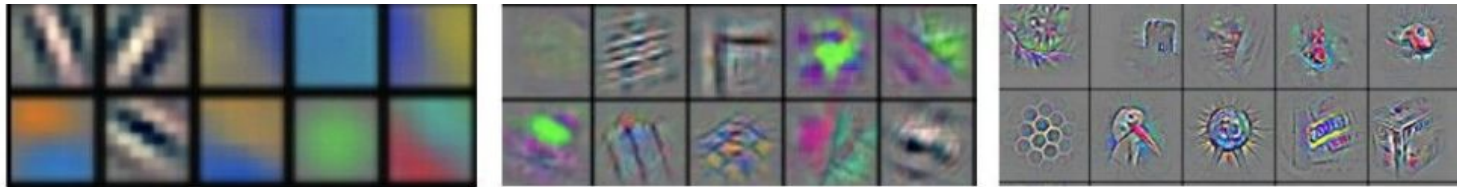
Further analysis of CNN structure



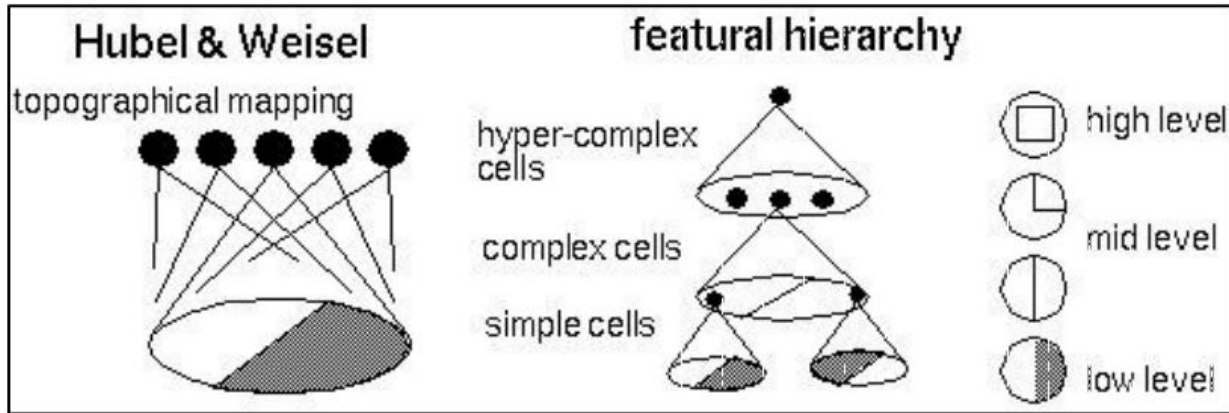
- Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization <https://arxiv.org/pdf/1610.02391>



Convolutional layer and visual cortex



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]



Convolutional layer and visual cortex



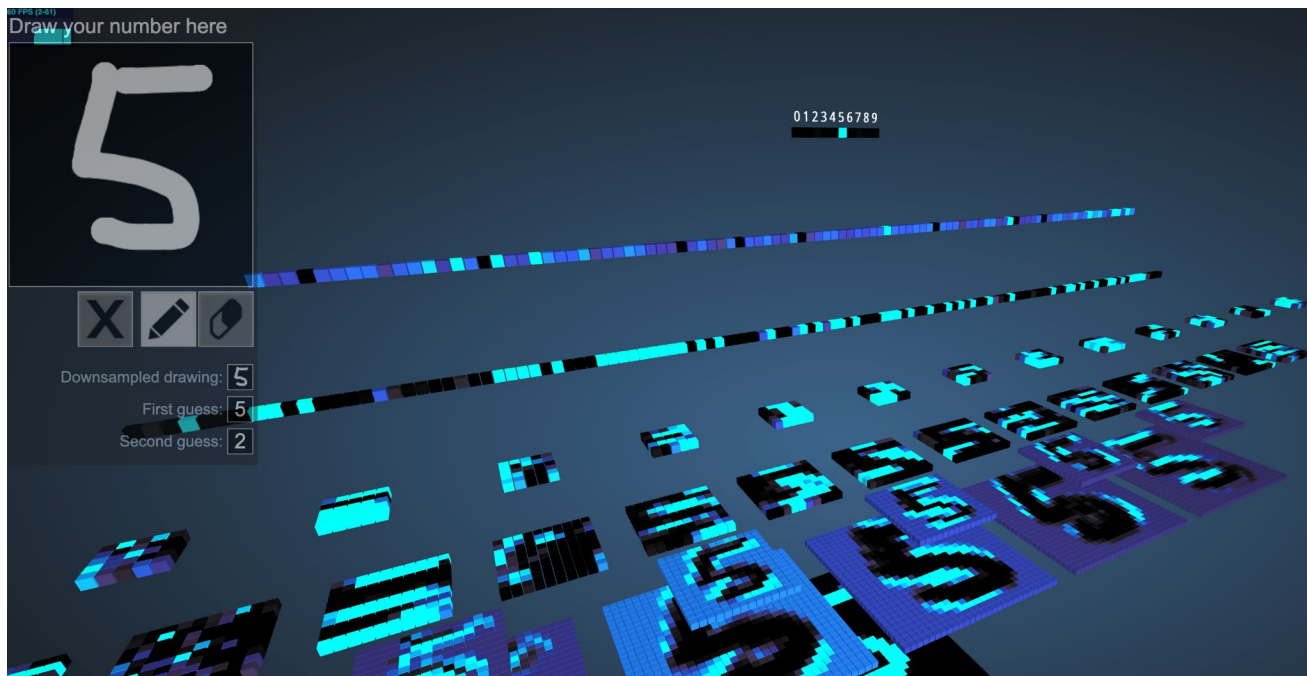
source

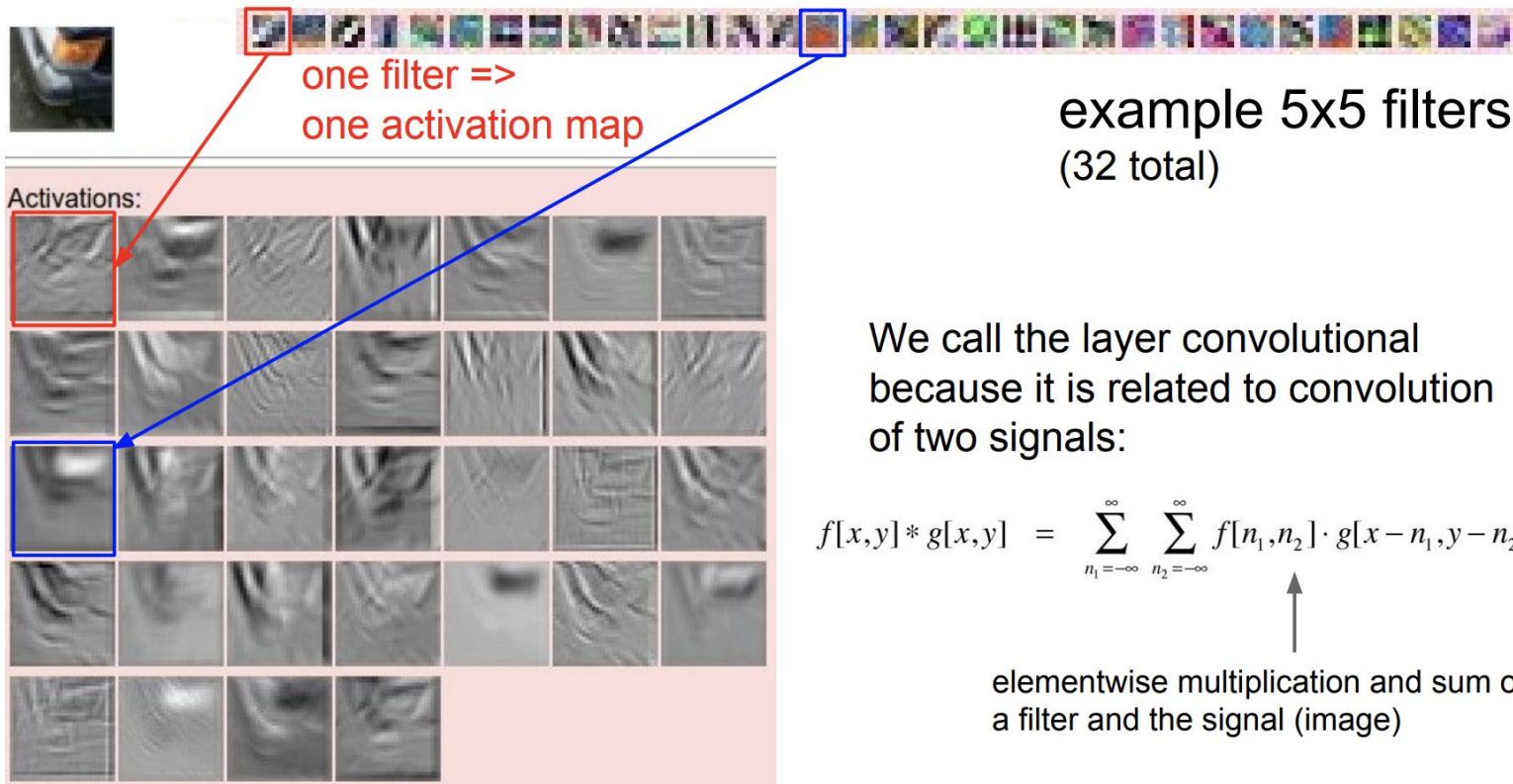
MNIST demo

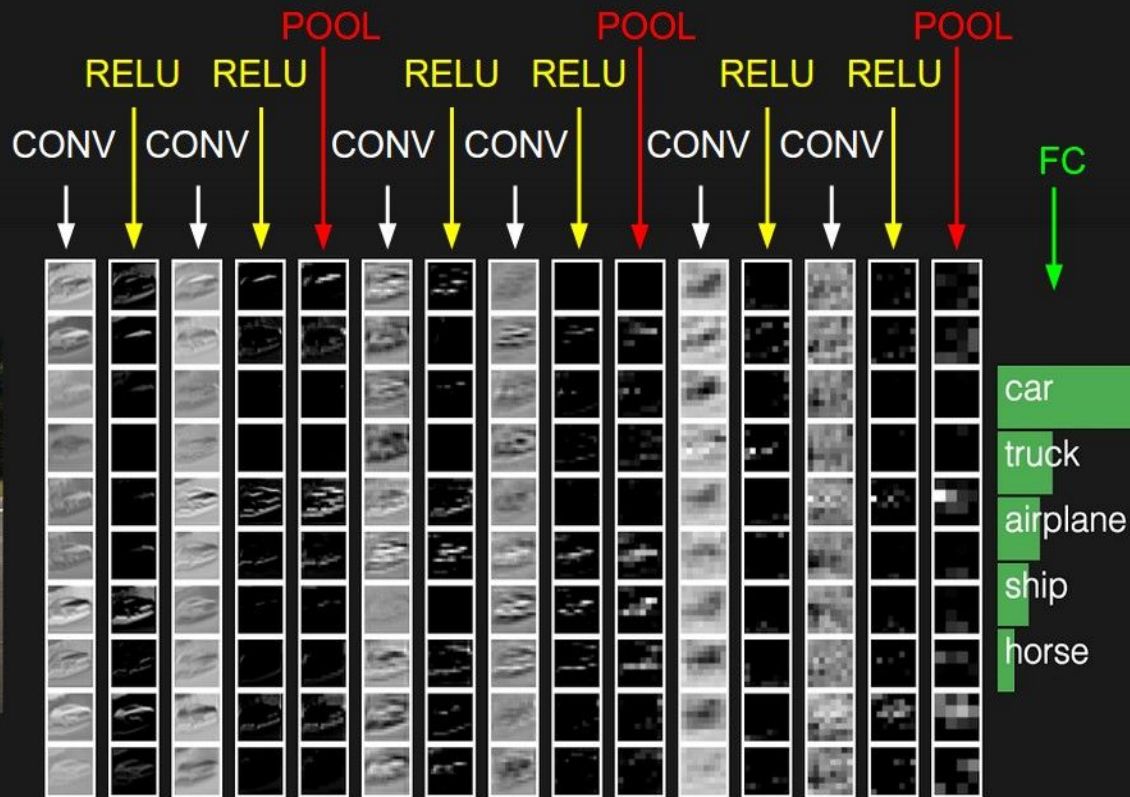


3D demo: https://adamharley.com/nn_vis/cnn/3d.html

2D demo (lightweight): https://adamharley.com/nn_vis/cnn/2d.html

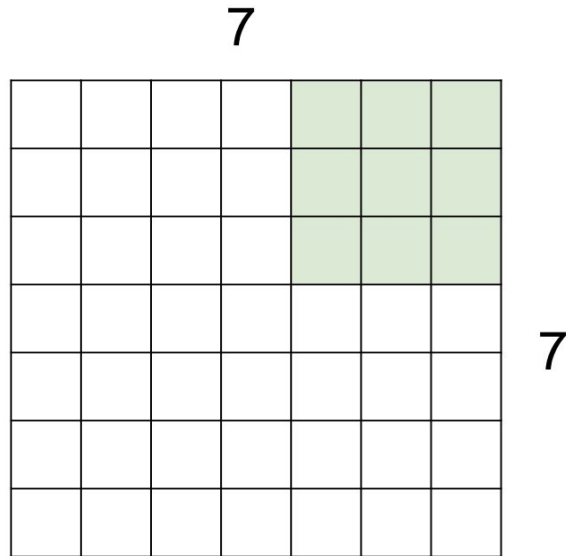








A closer look at spatial dimensions:

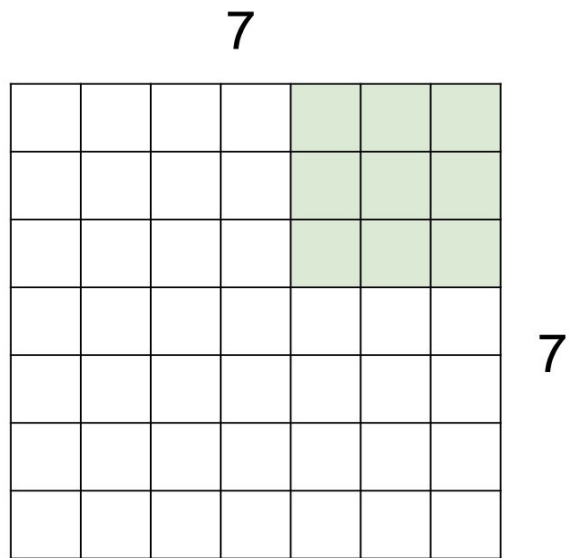


7x7 input (spatially)
assume 3x3 filter

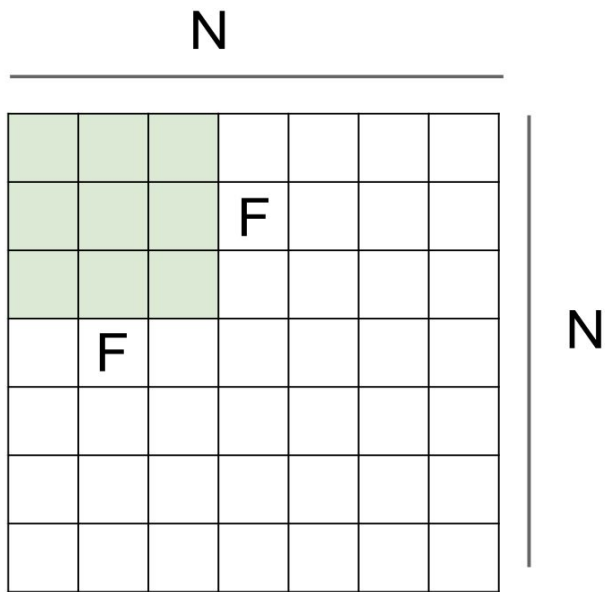
=> 5x5 output



A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!



Output size:
 $(N - F) / \text{stride} + 1$

e.g. $N = 7, F = 3$:

stride 1 $\Rightarrow (7 - 3)/1 + 1 = 5$

stride 2 $\Rightarrow (7 - 3)/2 + 1 = 3$

stride 3 $\Rightarrow (7 - 3)/3 + 1 = 2.33 \therefore \backslash$



In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

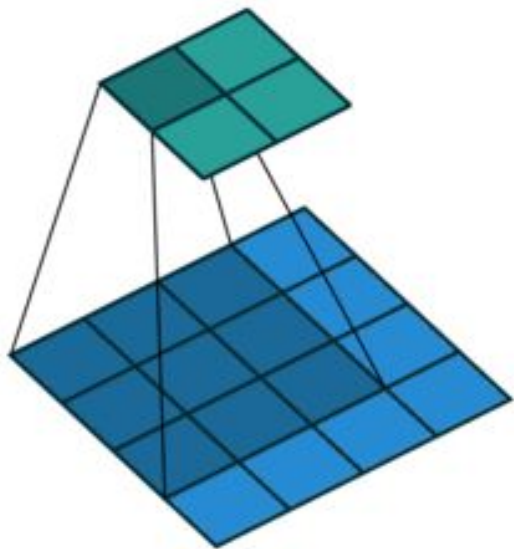
in general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F-1)/2$. (will preserve size spatially)

e.g. $F = 3 \Rightarrow$ zero pad with 1

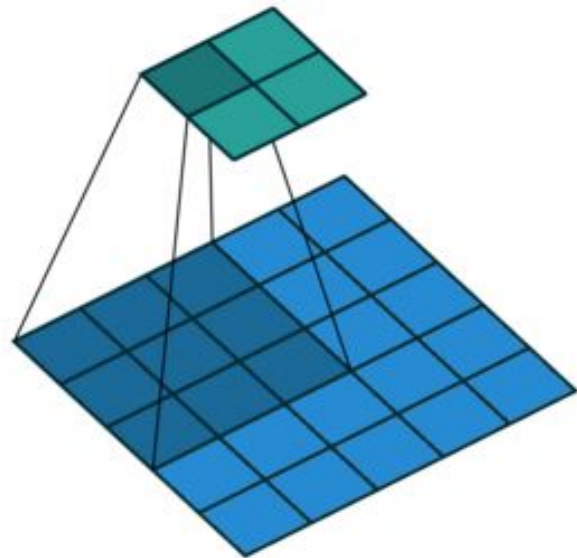
$F = 5 \Rightarrow$ zero pad with 2

$F = 7 \Rightarrow$ zero pad with 3

Strides, padding

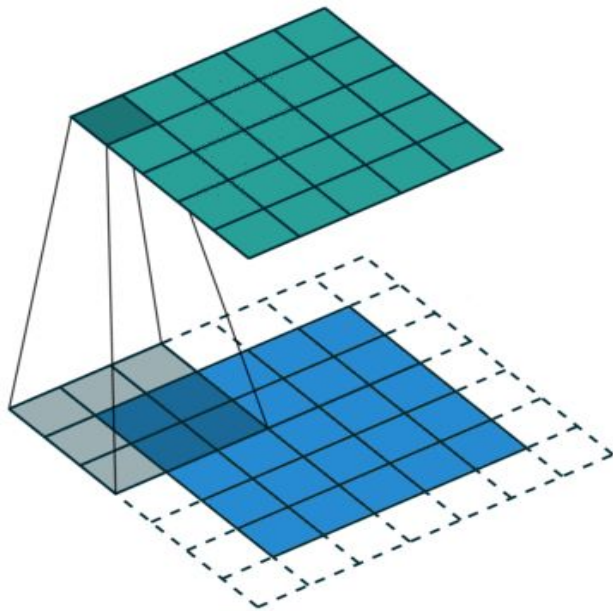


No padding, no strides

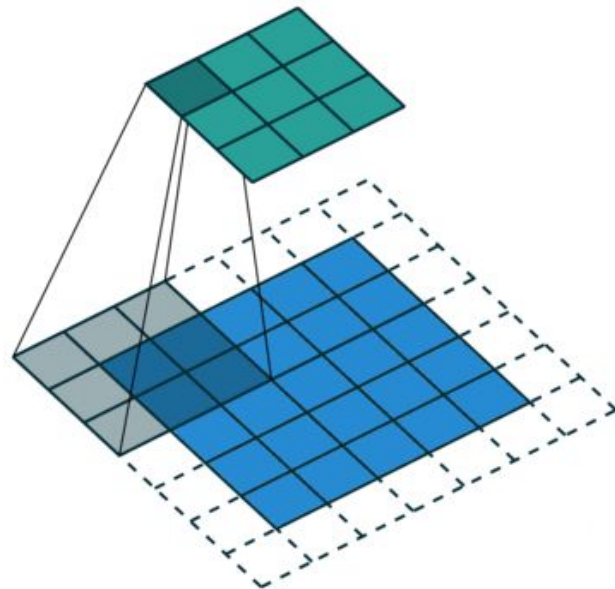


No padding, with strides

Strides, padding



With padding, no strides



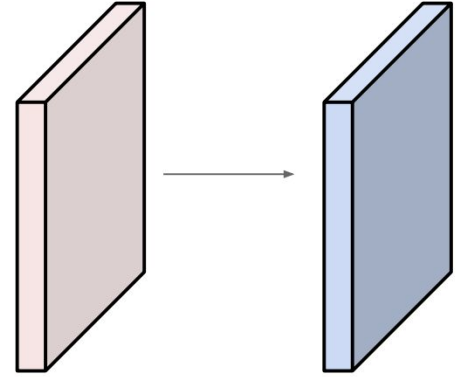
With padding, with strides

Examples time:

Input volume: **32x32x3**

10 5x5 filters with stride 1, pad 2

Output volume size: ?



Examples time:

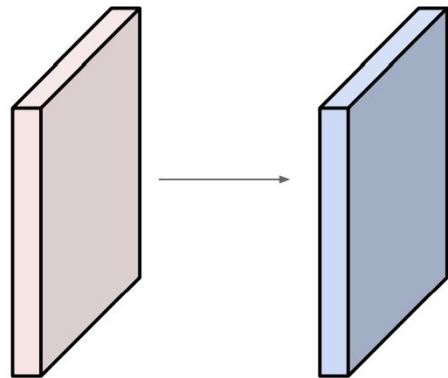
Input volume: **32x32x3**

10 **5x5** filters with stride **1**, pad **2**

Output volume size:

$(32 + 2 * 2 - 5) / 1 + 1 = 32$ spatially, so

32x32x10

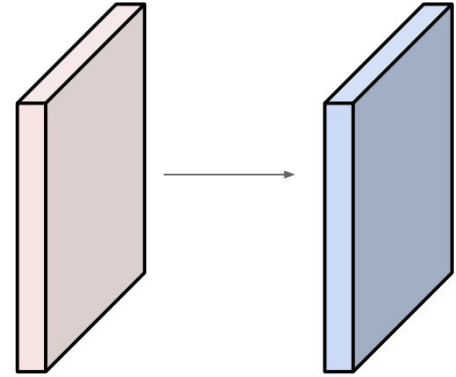


Examples time:

Input volume: **32x32x3**

10 5x5 filters with stride 1, pad 2

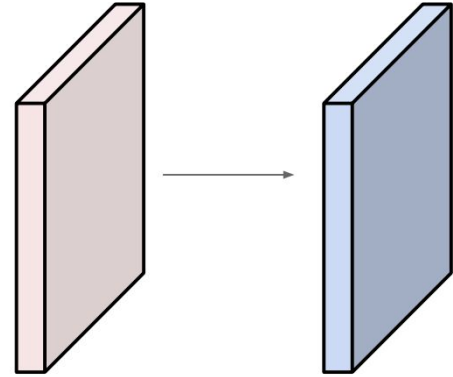
Number of parameters in this layer?



Examples time:

Input volume: **32x32x3**

10 **5x5** filters with stride 1, pad 2

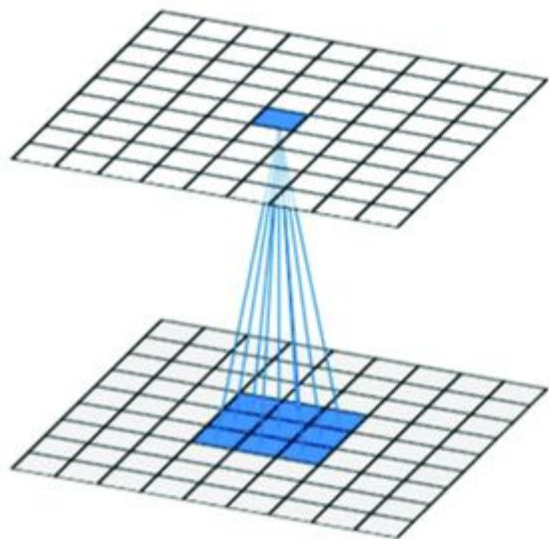


Number of parameters in this layer?

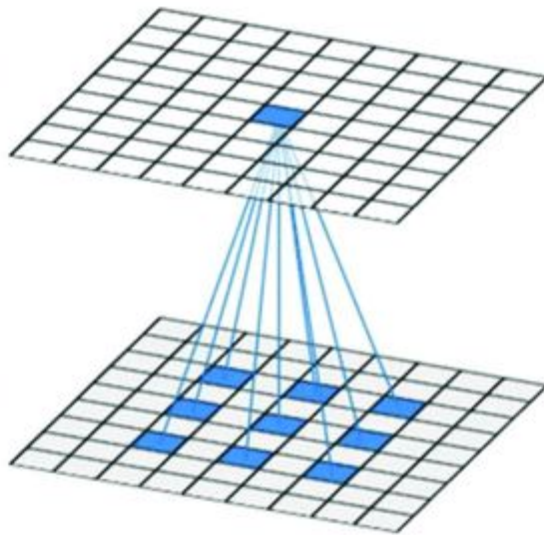
each filter has $5*5*3 + 1 = 76$ params (+1 for bias)

=> $76*10 = 760$

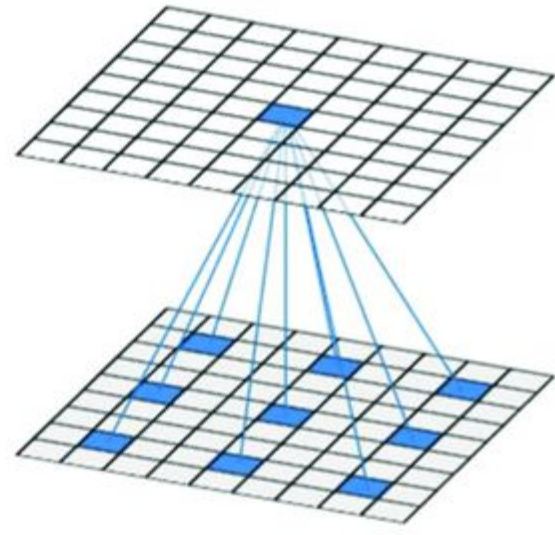
Dilation



dilation=1

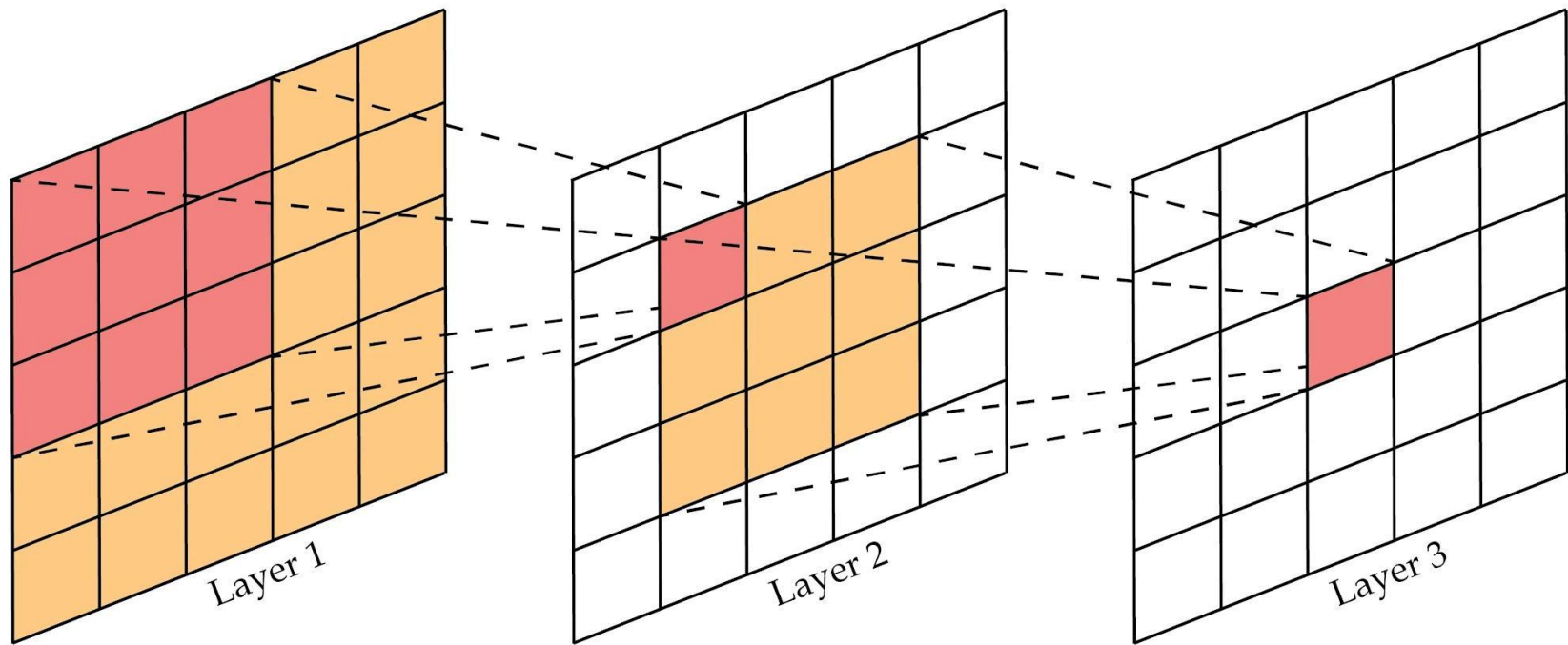


dilation=2



dilation=3

Receptive field





Compare parameters number and receptive field of 3x3 and 5x5

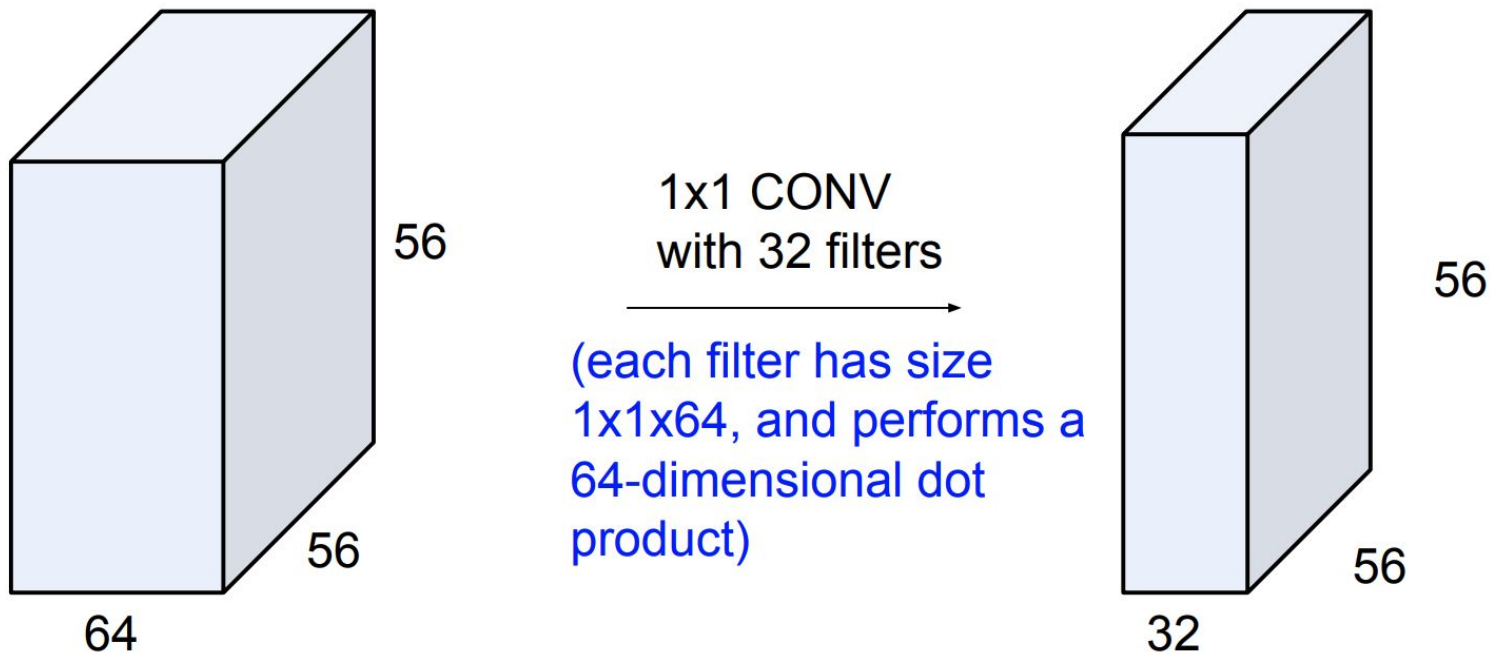
3x3 is better because of receptive field and number of parameters

1. 1 * 3x3: RF=3; P=10
2. 1 * 5x5: RF=5; P=26
3. 2 * 3x3: RF=5; P=20

2x2?; 2 * 2x2: RF=3; P=10

4x4?

Pointwise (1x1) Convolution



Spatial Separable Convolution

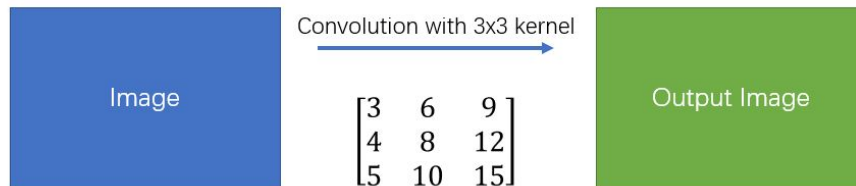


$$\begin{bmatrix} 3 & 6 & 9 \\ 4 & 8 & 12 \\ 5 & 10 & 15 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \times [1 \quad 2 \quad 3]$$

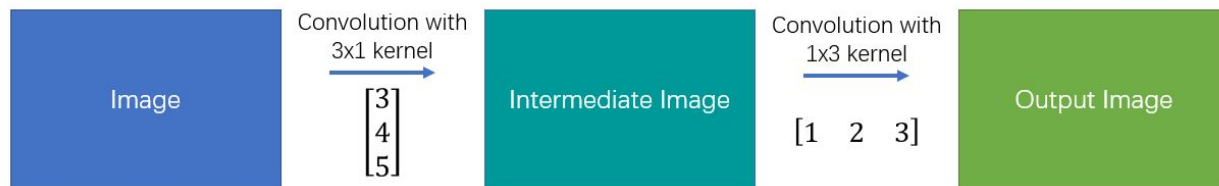
Spatial Separable Convolution



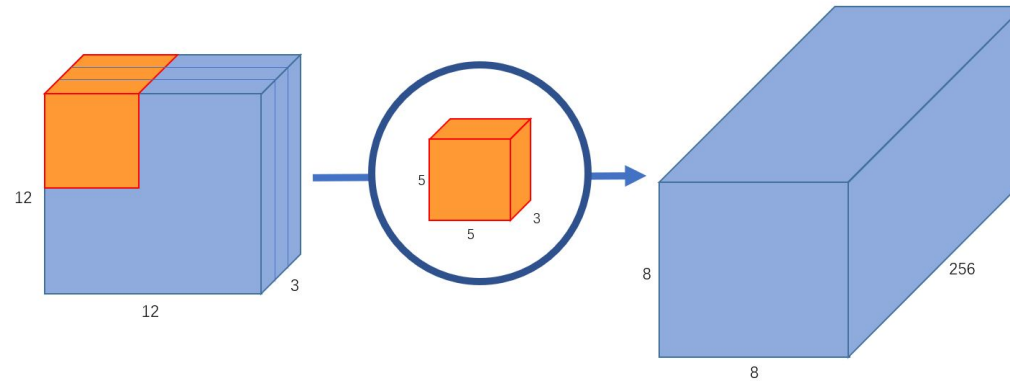
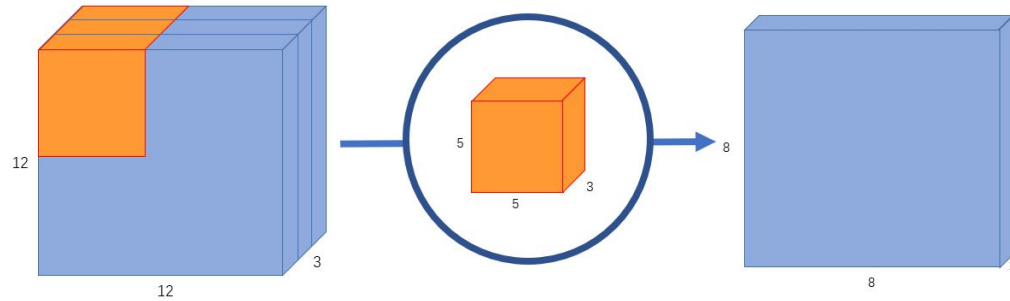
Simple Convolution



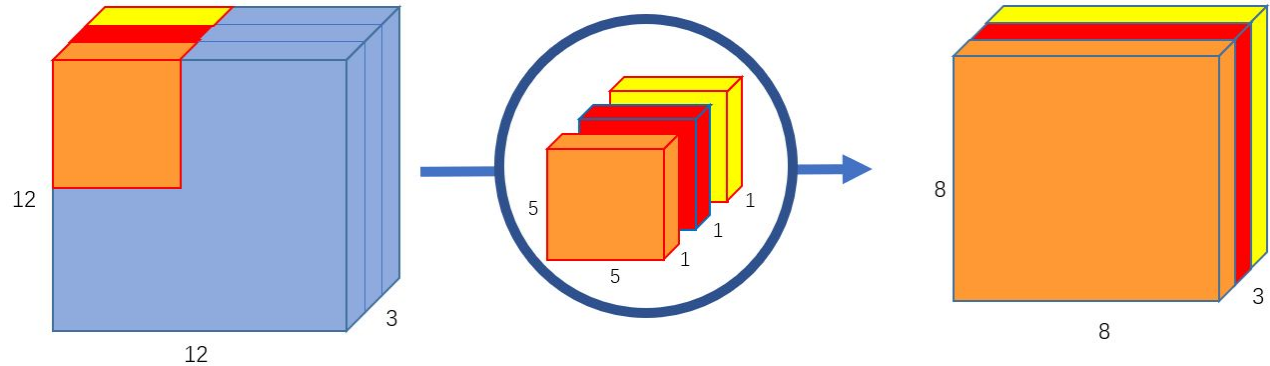
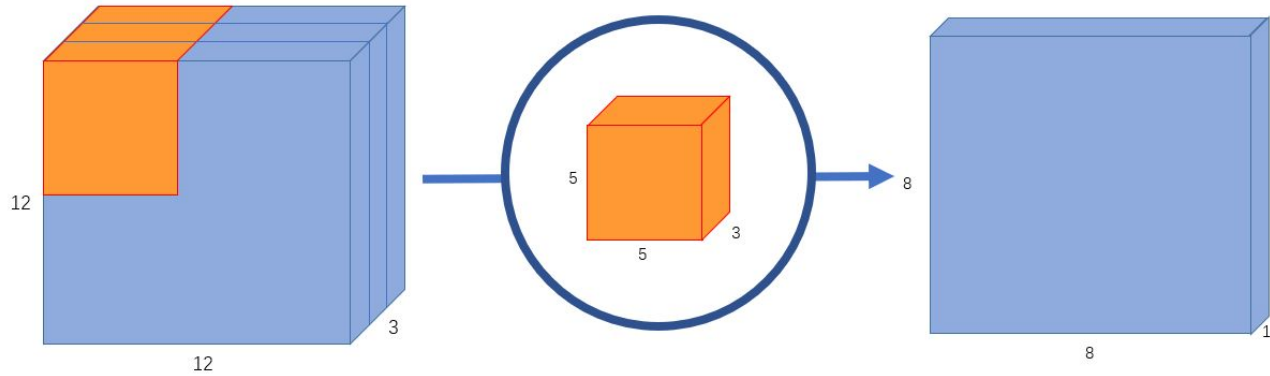
Spatial Separable Convolution



Normal Convolution



Depthwise Convolution





Conv layer hyperparams

- stride
- padding
- dilation
- groups

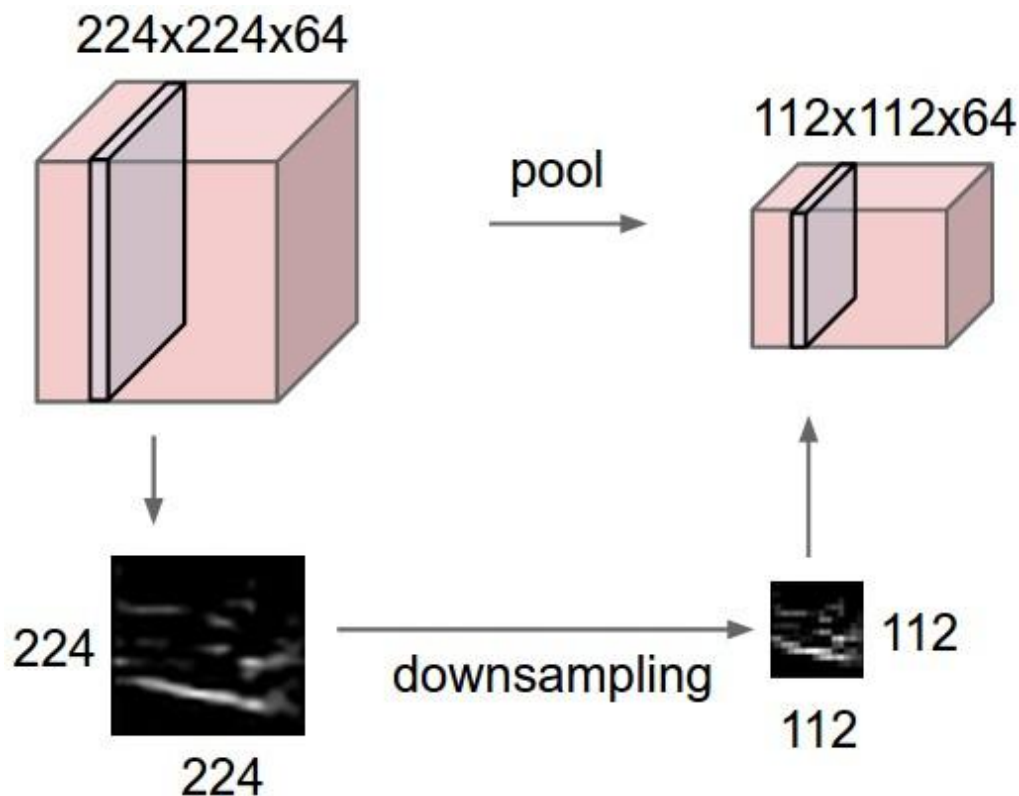
<https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

Pooling layer

girafe
ai

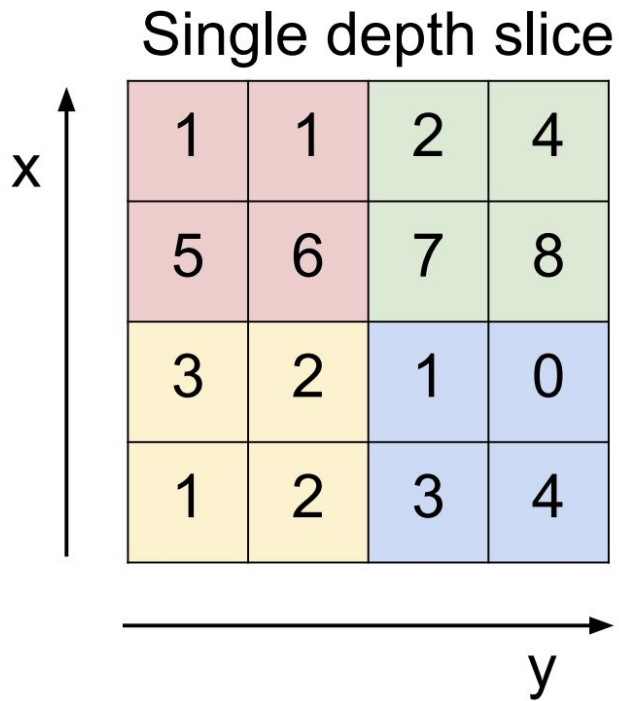
02

Pooling layer



- Makes the representations smaller and more manageable
- Operates over each channel independently
- No trainable params, so doesn't overfit

Max pooling



max pool with 2x2 filters
and stride 2



6	8
3	4

Average pooling



2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Average Pool



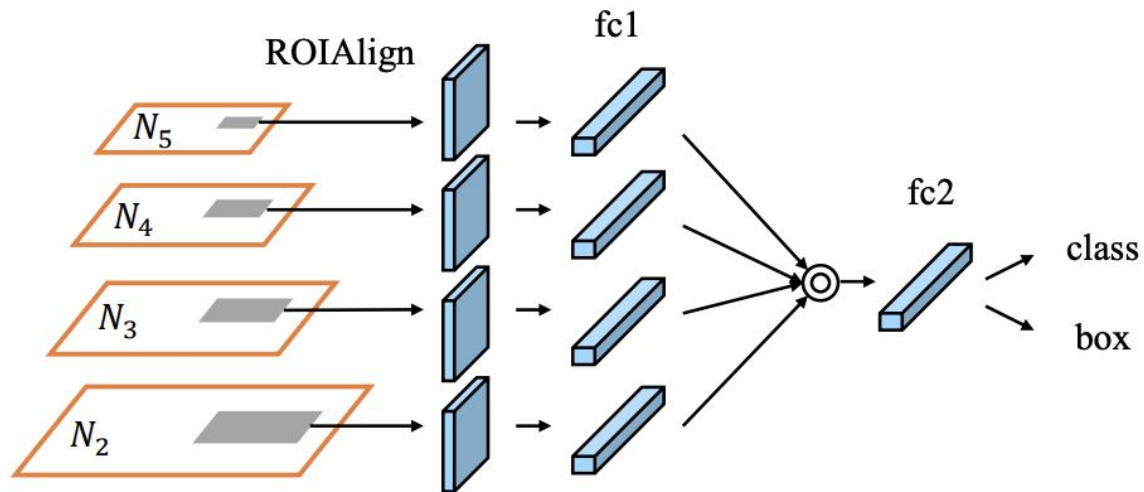
Filter - (2 x 2)
Stride - (2, 2)

4.25	4.25
4.25	3.5

Adaptive pooling



- Pools adaptive region size into fixed output size
- Fixed size is required for following linear layers
- Allows to put arbitrary image size into convolution network
- Both Max- and Average-



[`torch.nn.AdaptiveAvgPool2d`](#)

Classification architectures overview

girafe
ai

03

Image classification then



2007: Asirra

CAPTCHA that asks users to identify cats out of a set of 12 photographs of both cats and dogs.

Solved by **humans in 99.6%** of cases.

Barring a major advance in machine vision, we expect **computers** will have no better than a **1/54,000 chance** of solving it

[Elson et al](#)



Image classification now



2014: Asirra on kaggle

[Competition page](#)

#	△	Team	Members	Score	Entries	Last	Solution
1	—	Pierre Sermanet		0.98914	5	10y	
2	▲ 4	orchid		0.98308	17	10y	
3	—	Owen		0.98171	15	10y	
4	—	Paul Covington		0.98171	3	10y	
5	▼ 3	Maxim Milakov		0.98137	24	10y	
6	▼ 1	we've been in KAIST	 	0.98102	8	10y	
7	▲ 1	Doug Koch		0.98057	6	10y	
8	▲ 2	fastml.com/cats-and-dogs		0.98000	6	10y	

Last Submission: 02/02/2014, 1:43:19 GMT+4

IMAGENET competition



- ImageNet Large Scale Visual Recognition Challenge (ILSVRC) since 2010
- 14.2m images
- 1000 classes
- <http://image-net.org/explore>

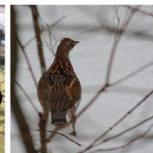
ILSVRC



flamingo



cock



ruffed grouse



quail



partridge

...



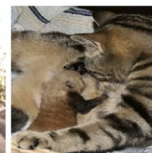
Egyptian cat



Persian cat



Siamese cat



tabby



lynx

...

ballplayer 69.22%



ice_cream 99.60%



anemone_fish 92.48%



lemon 97.06%



African_elephant 89.94%



magnetic_compass 97.08%



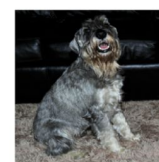
forklift 98.95%



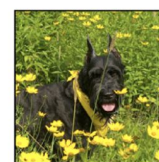
ice_bear 84.80%



miniature schnauzer



standard schnauzer



giant schnauzer

...

IMAGENET dataset

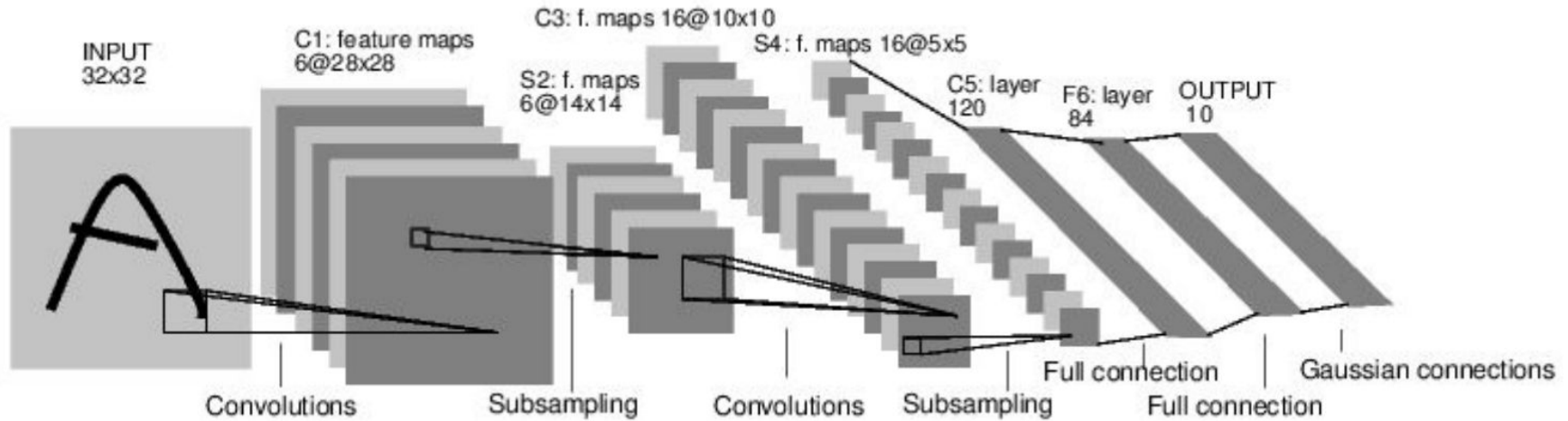


- Most of classes are dogs
- Most correlated classes are chihuahua and cupcake

Чтобы подтвердить, что вы не робот, отметьте
КЕКСЫ

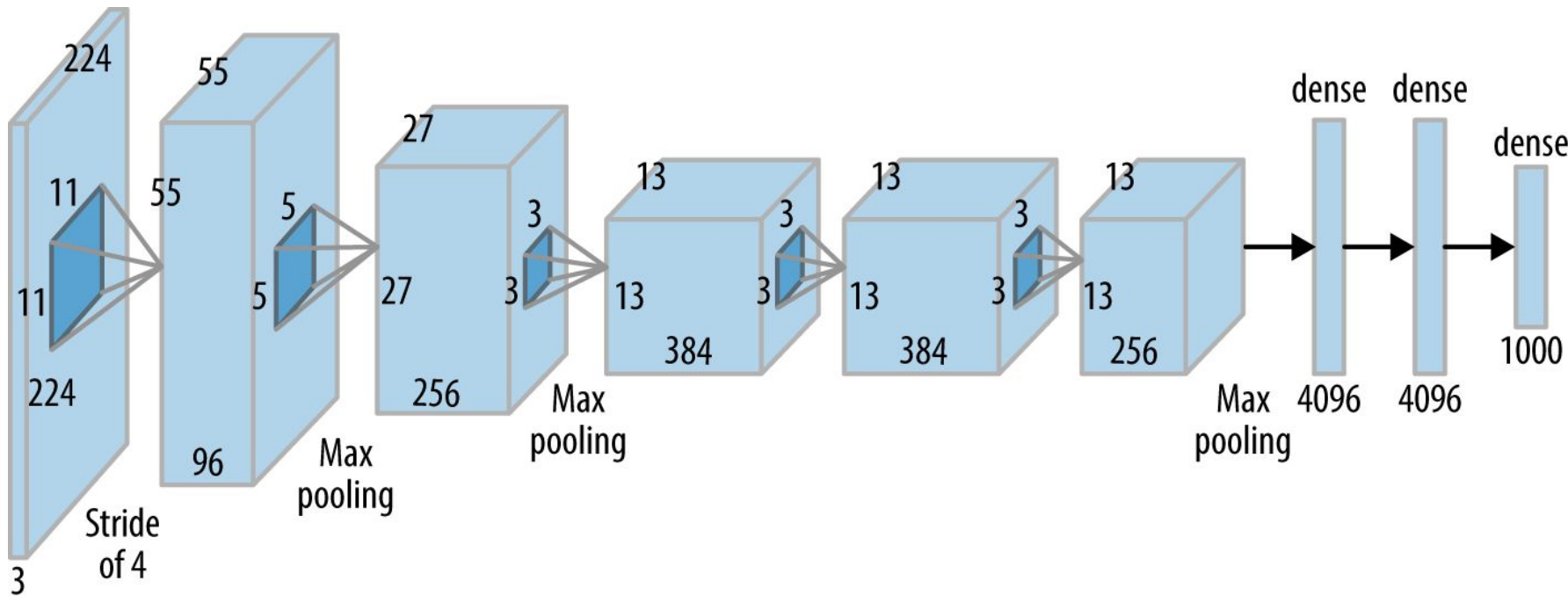


LeNet-5



[LeNet-5, LeCun 1998]

AlexNet

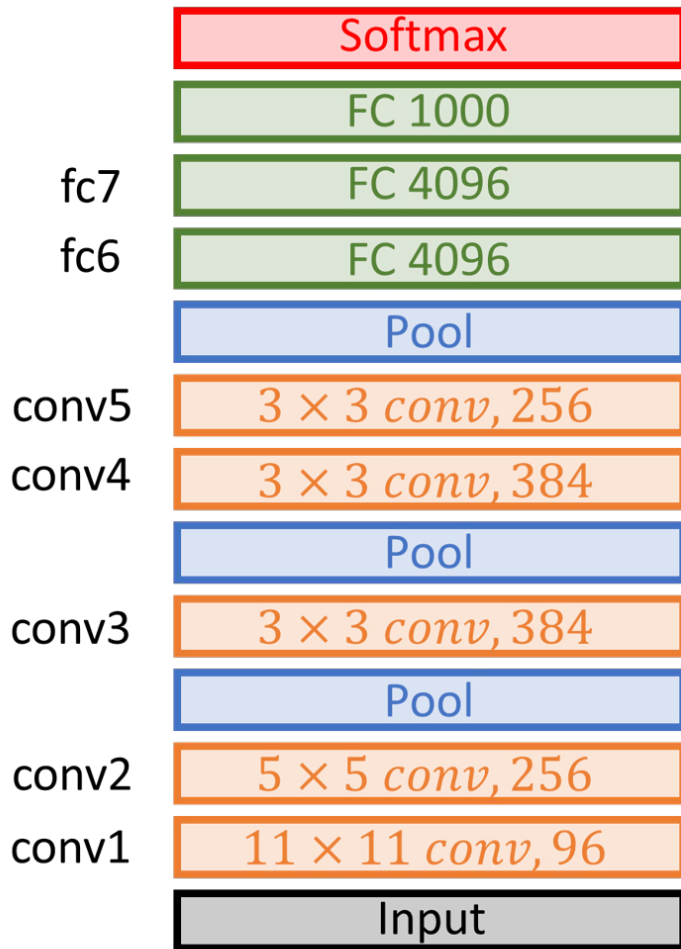


AlexNet

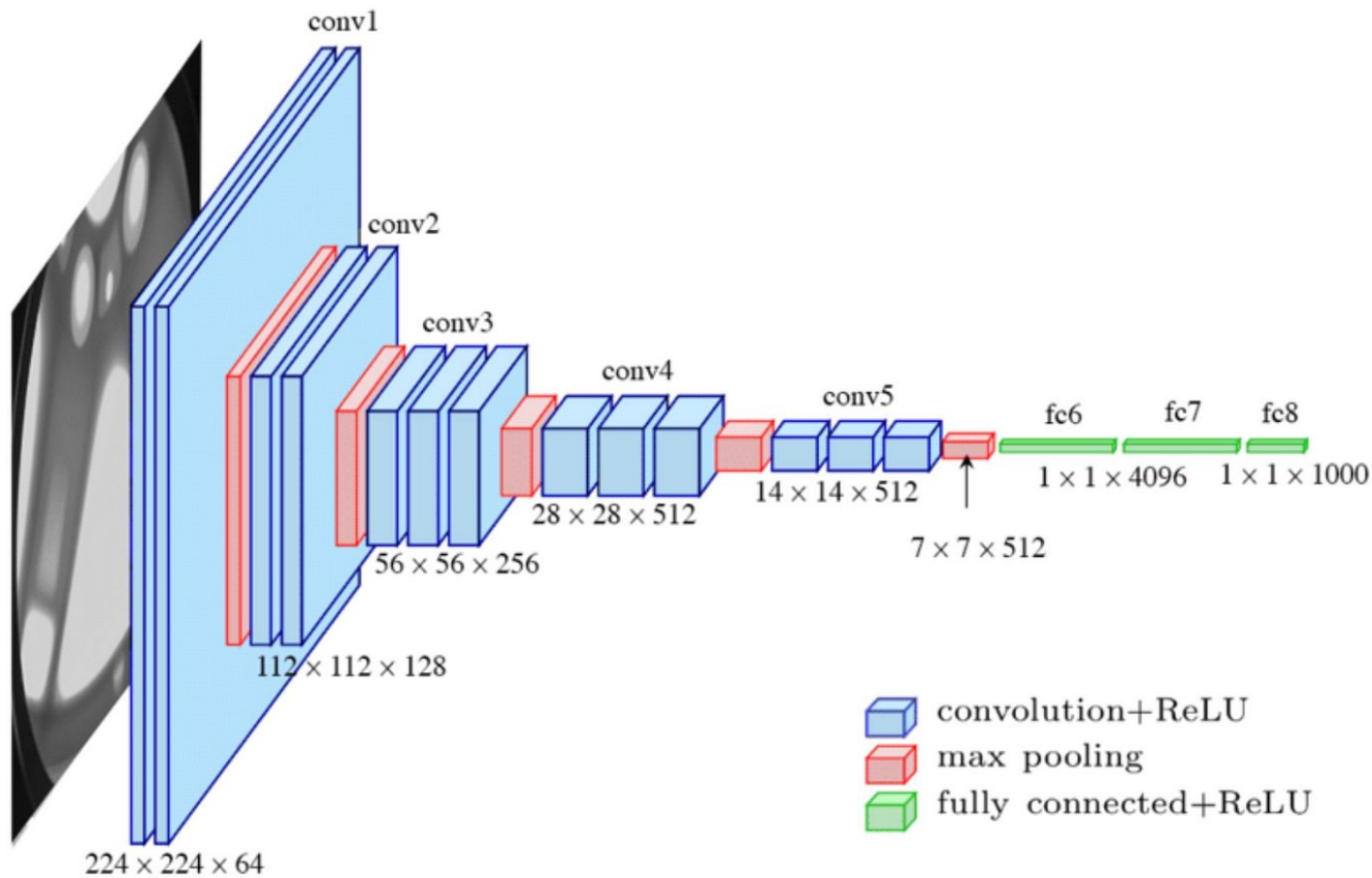
- Krizhevsky et al. 2012
- Used 2 GPUs for training
- First win of ImageNet competition with Deep Neural Networks
- Used big convolutions
- Fully connected at the end
- 7 layers

Takeouts:

- First use of ReLU
- Increasing number of channels
- 16.4%

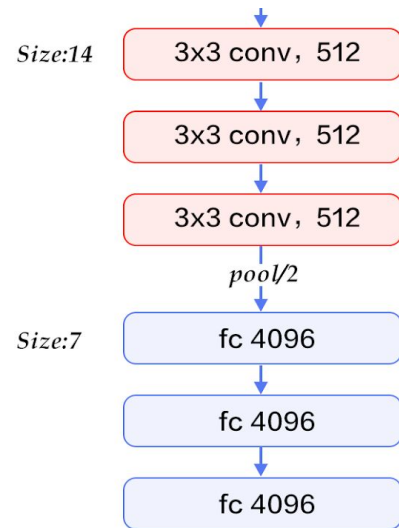
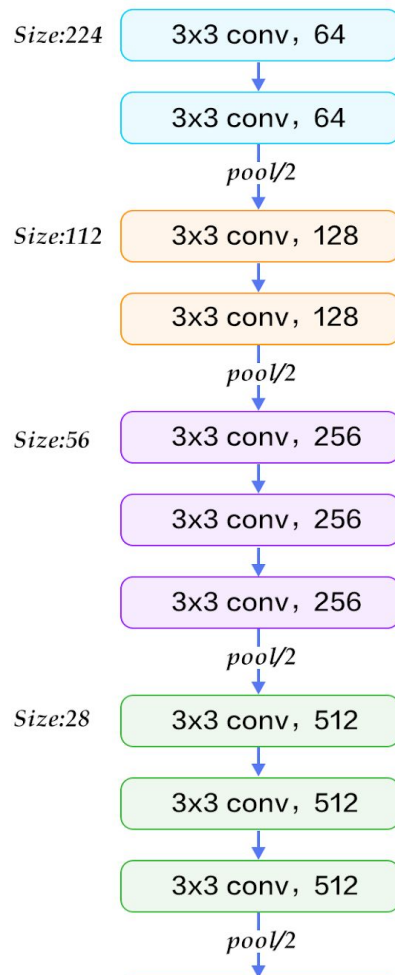


VGGNet

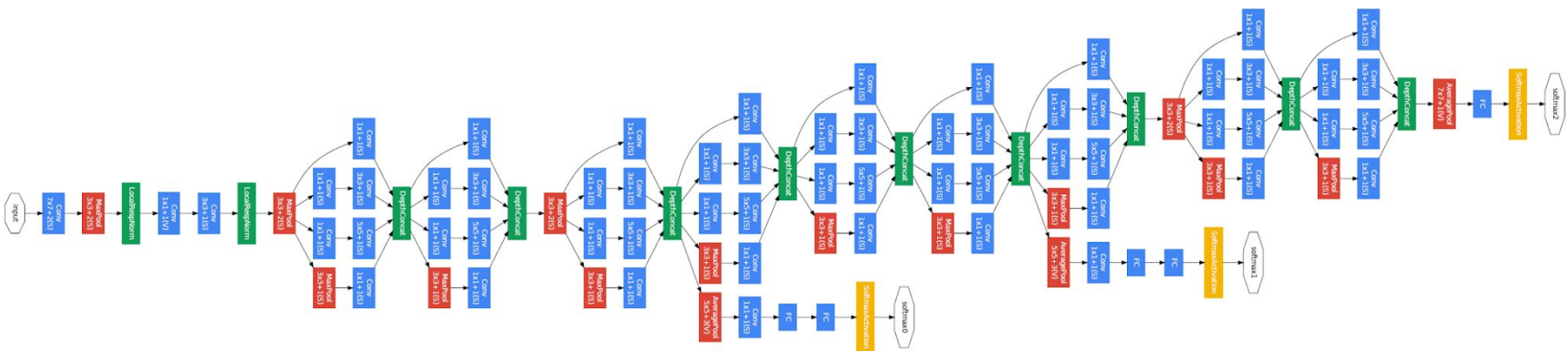


VGGNet

- Simonyan, Zisserman 2014
- All convolutions are 3x3
- Exponentially increasing channels
- 19 layers
- Requires a lot of memory
- Gradient vanishes =(



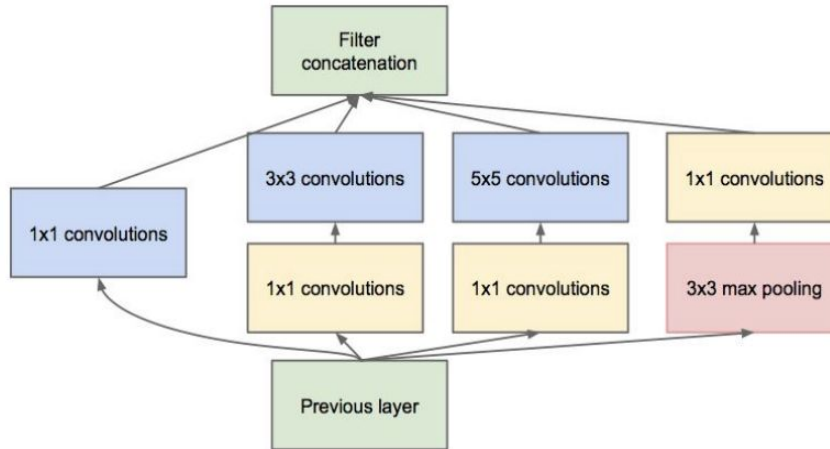
GoogLeNet



Inception module

[Szegedy et al., 2014]

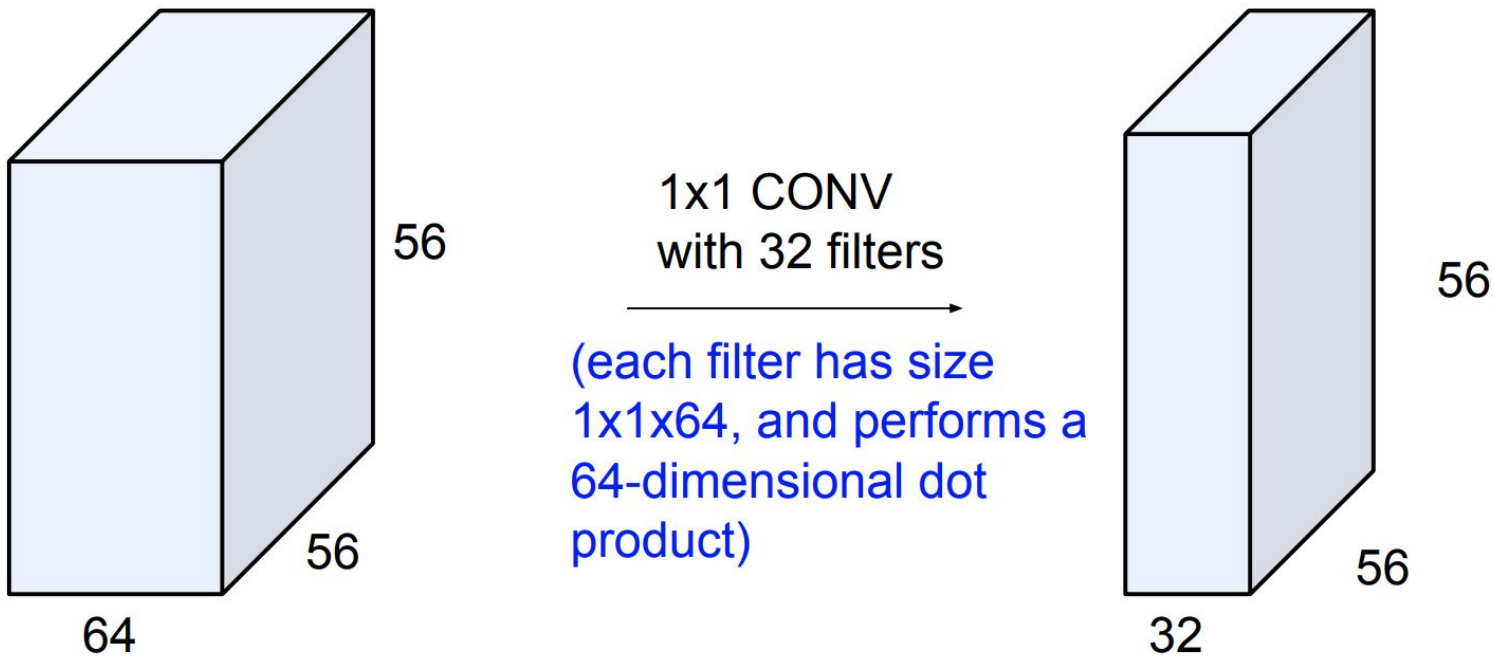
GoogLeNet



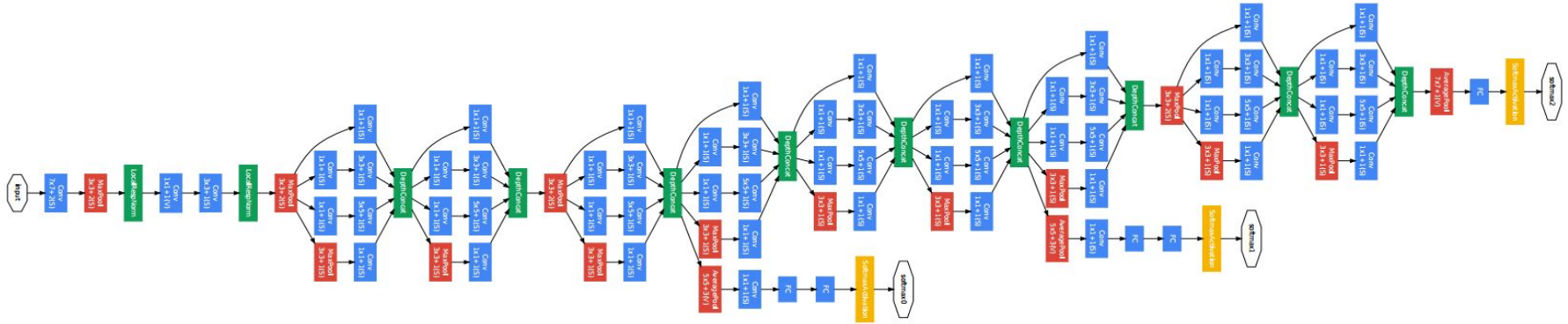
Inception module



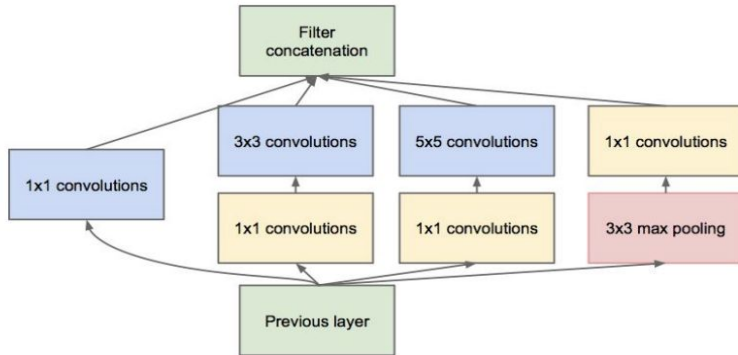
Once again: 1x1 convolutions



GoogLeNet



[Szegedy et al., 2014]



Inception module

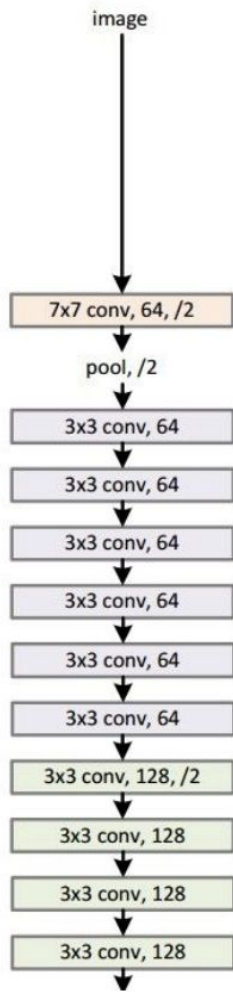
ILSVRC 2014 winner (6.7% top 5 error)



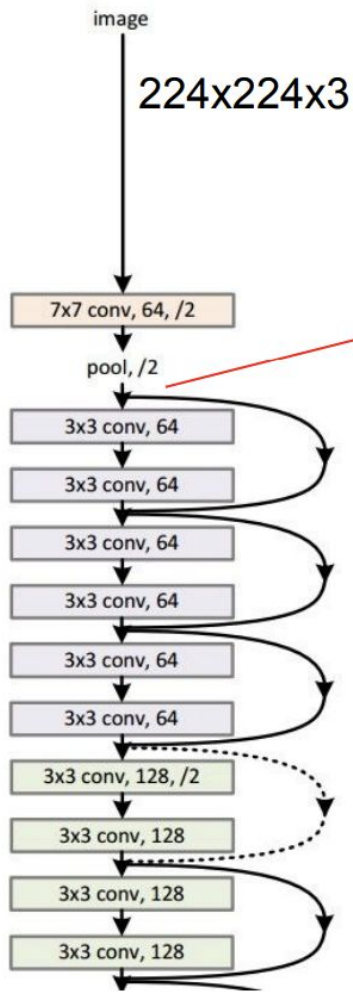
ResNet

spatial dimension
only 56x56!

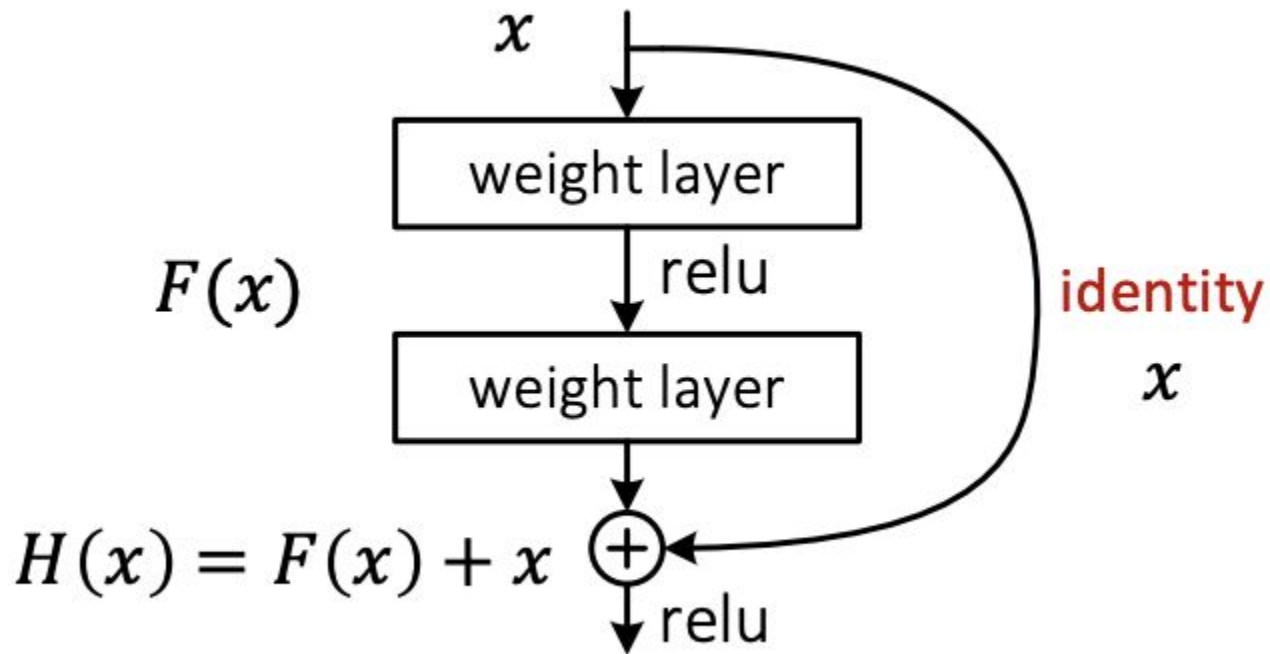
34-layer plain



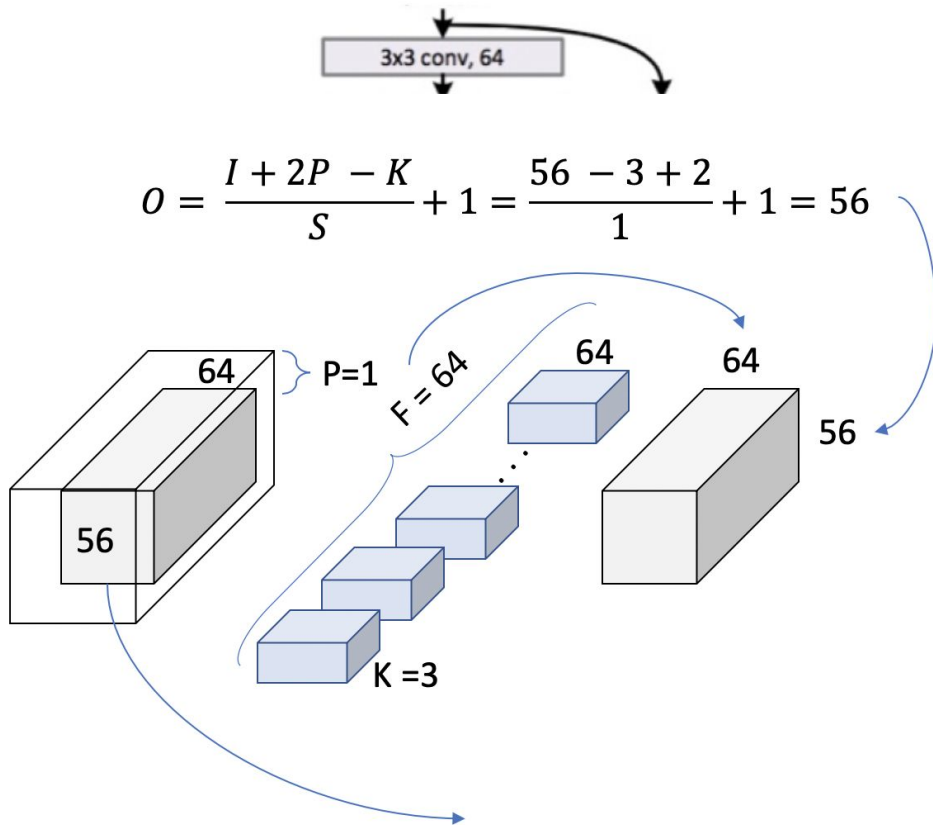
34-layer residual



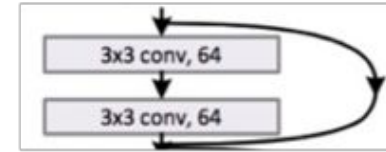
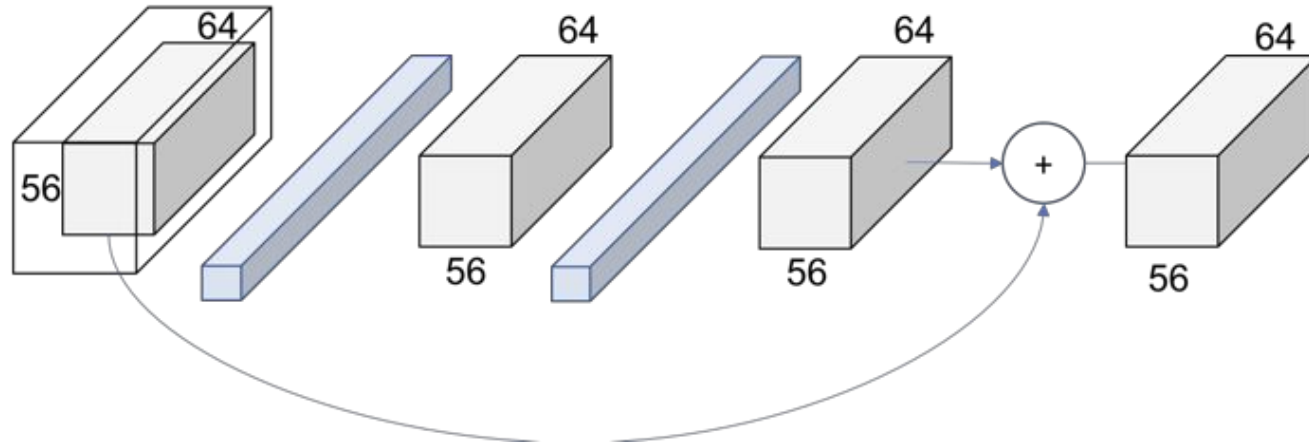
Residual Block



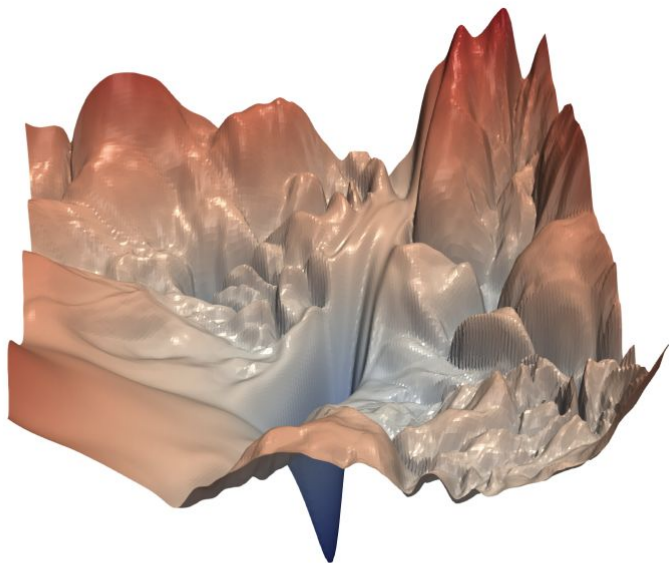
Residual Block



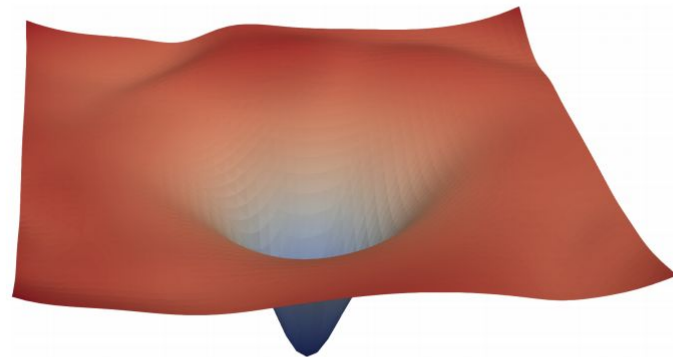
Residual Block



Residual Block



(a) without skip connections

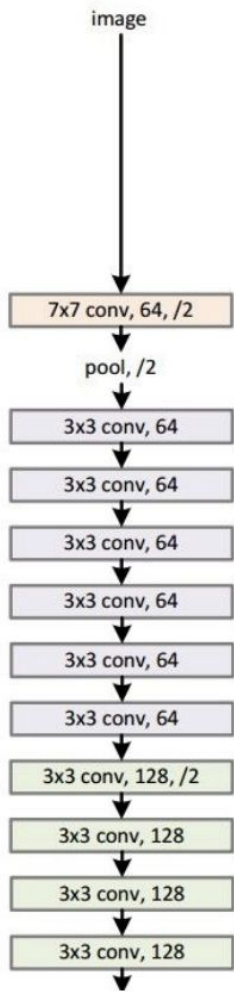


(b) with skip connections

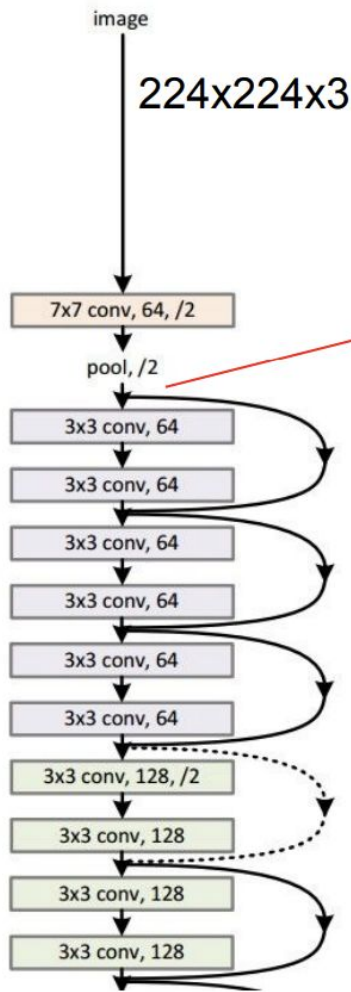
Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.



34-layer plain



34-layer residual



224x224x3

[He et al., 2015]

**spatial dimension
only 56x56!**

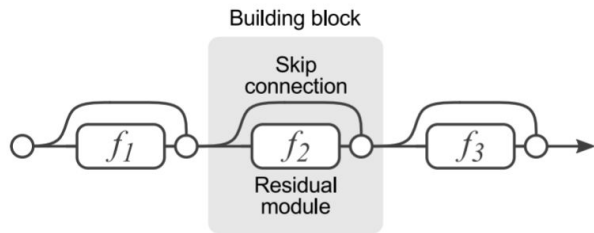
- Batch Normalization after every CONV layer
- Xavier/2 initialization from He et al.
- SGD + Momentum (0.9)
- Learning rate: 0.1, divided by 10 when validation error plateaus
- Mini-batch size 256
- Weight decay of 1e-5
- No dropout used

ILSVRC 2015 winner (3.6% top 5 error)

ResNet is implicit ensemble

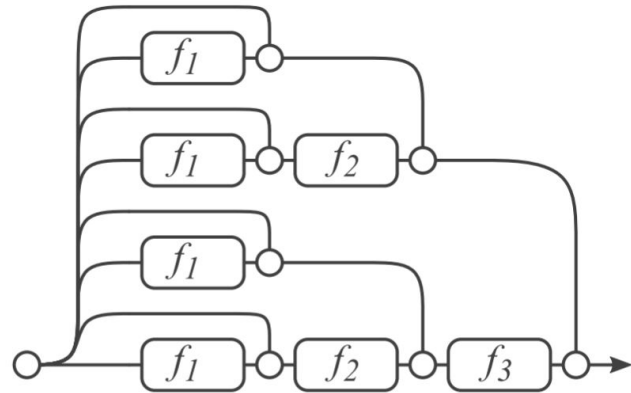


Residual Networks Behave Like Ensembles of Relatively Shallow Networks



(a) Conventional 3-block residual network

=



(b) Unraveled view of (a)

ResNeXt

Same as ResNet, but scalable

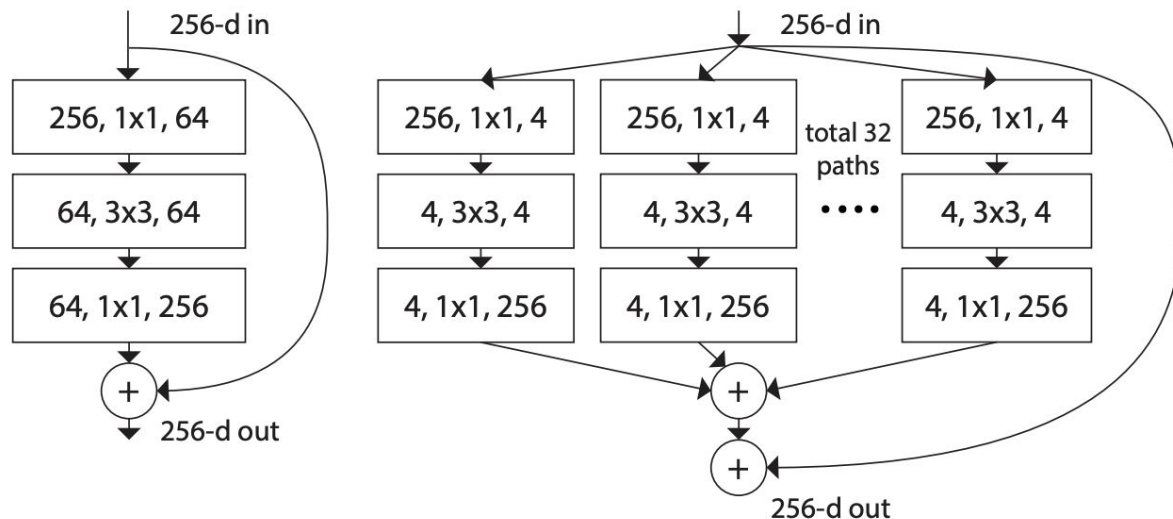
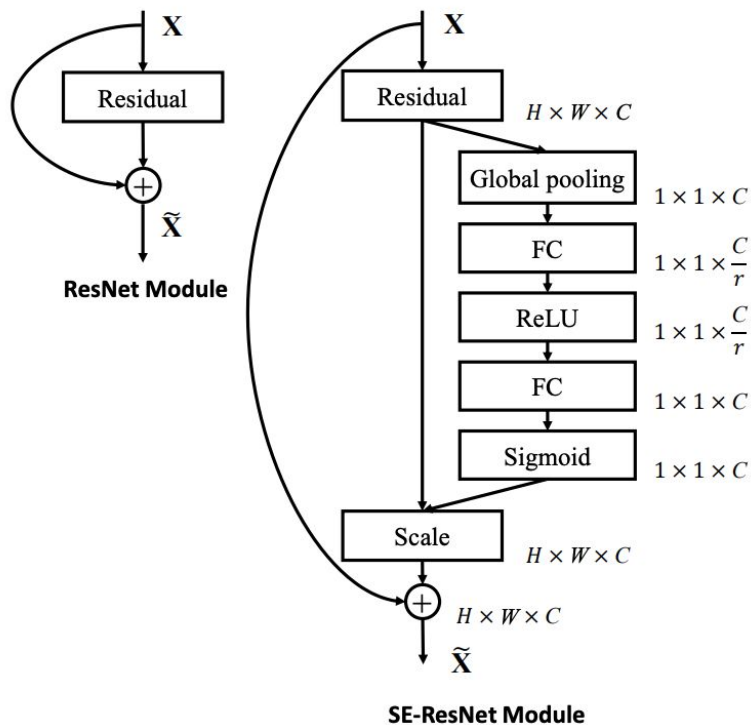


Figure 1. **Left:** A block of ResNet [14]. **Right:** A block of ResNeXt with cardinality = 32, with roughly the same complexity. A layer is shown as (# in channels, filter size, # out channels).

SENet



Linear layer (no attention):

$$\mathbf{Z} = \mathbf{X} * \mathbf{W} + \mathbf{b}$$

$\mathbf{W}, \mathbf{b}$ - constants

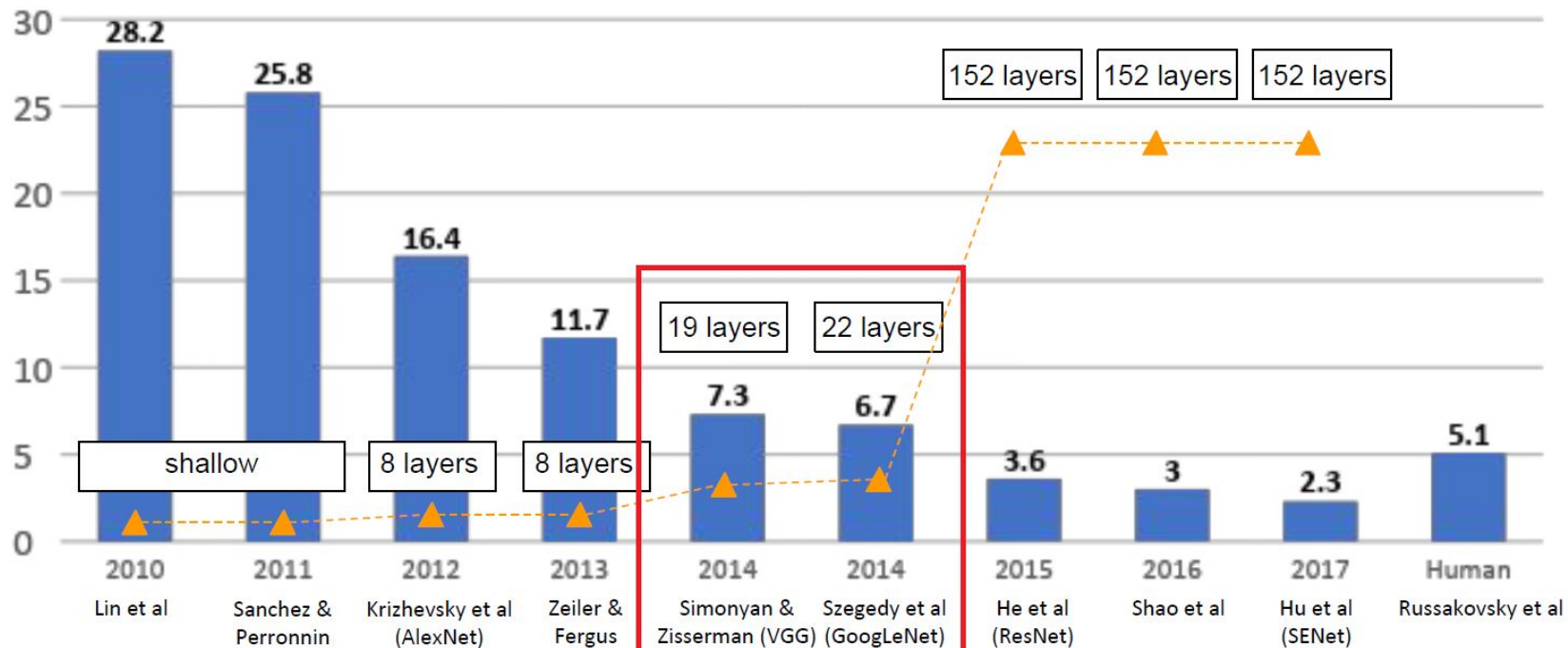
Attention:

$$\mathbf{Z} = \mathbf{X} * \mathbf{W}(\mathbf{X}) + \mathbf{b}(\mathbf{X})$$

$\mathbf{W}, \mathbf{b}$ - functions of $\mathbf{X}$

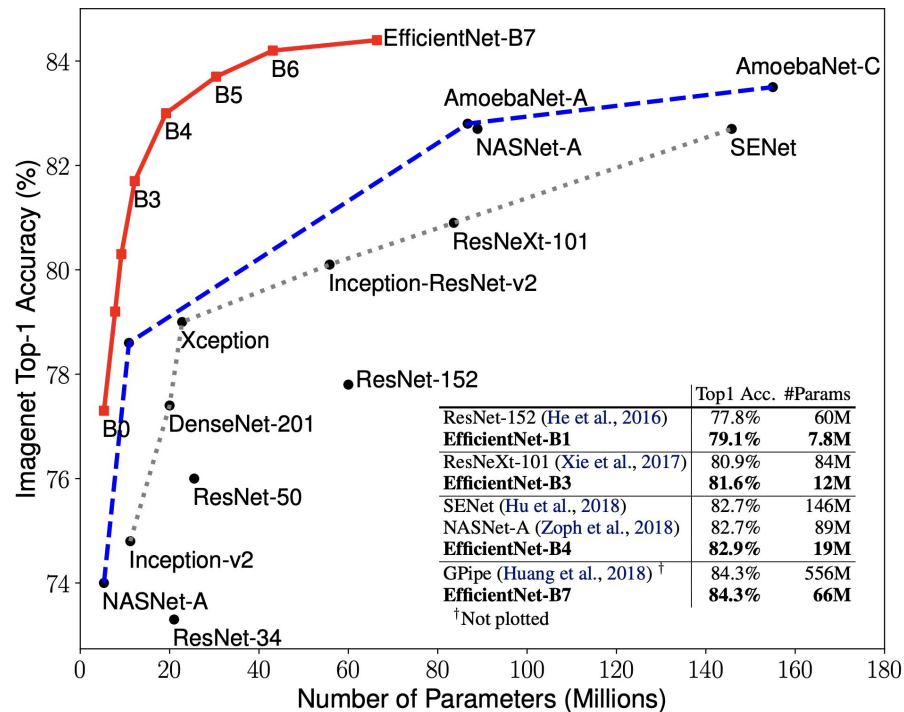
Fig. 3. The schema of the original Residual module (left) and the SE-ResNet module (right).

ImageNet summary



EfficientNet

<https://arxiv.org/pdf/1905.11946>



EfficientNet

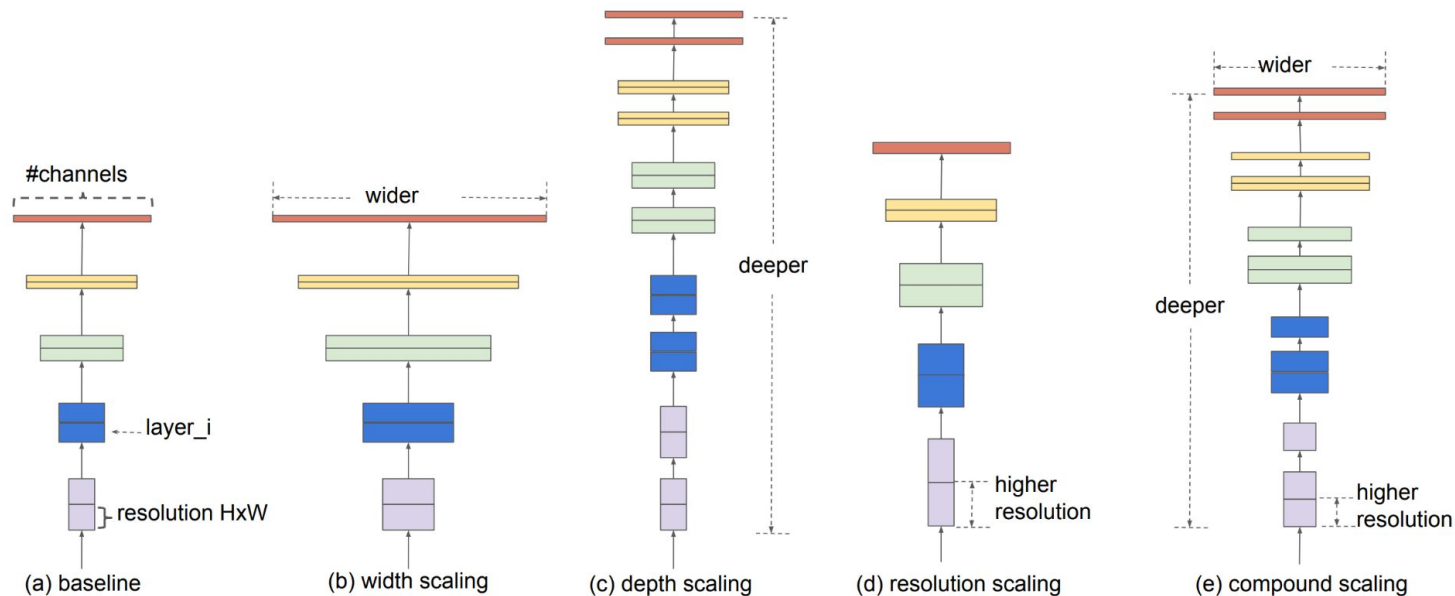


Figure 2. Model Scaling. (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

Vanishing gradient

Vanishing gradient is present in **all** deep neural network architectures.

- Due to chain rule / choice of nonlinearity function, gradient can become vanishingly small during backpropagation
- Lower levels are hard to train and are trained slower
- Solutions
 - skip-connections (ResNet)

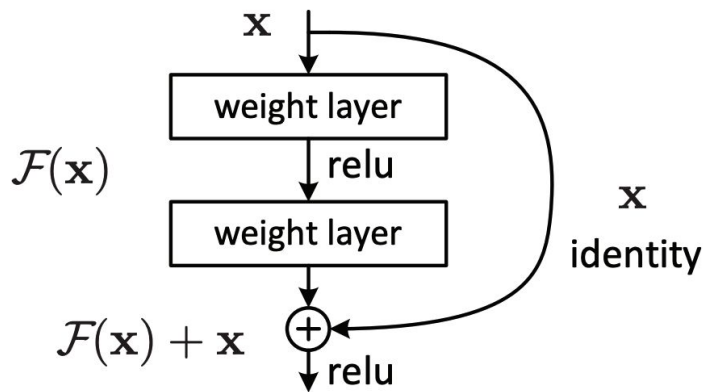


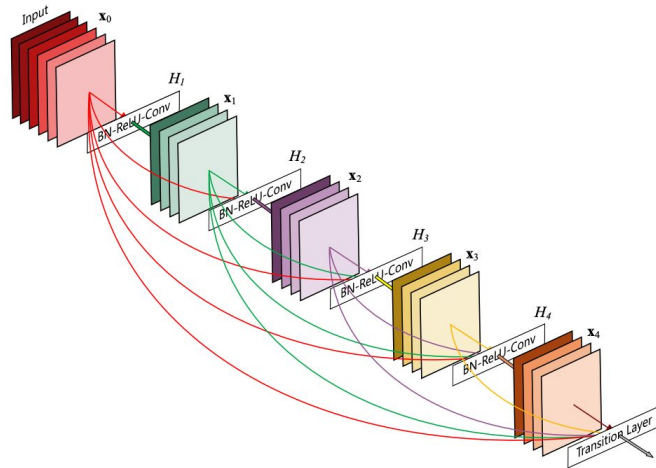
Figure 2. Residual learning: a building block.

Vanishing gradient



Vanishing gradient is present in **all** deep neural network architectures.

- Due to chain rule / choice of nonlinearity function, gradient can become vanishingly small during backpropagation
- Lower levels are hard to train and are trained slower
- Solutions
 - skip-connections (ResNet)
 - dense connections (DenseNet)



Summary



- ConvNets stack convolutional, pooling and dense layers
- Trend towards smaller filters and deeper architectures
- 1×1 convolutions are meaningful
- Humanity is already beaten on ImageNet.

Revise



1. Convolutional layer structure
2. Pooling layers
3. Top architectures overview

Thanks for attention!

Questions?

